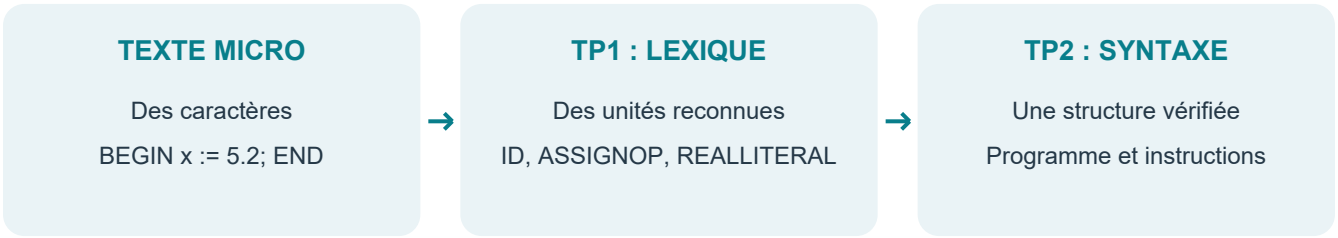
COMPRENDRE LA COMPILATION

Le cours, puis les TP 1 et 2 expliqués depuis le début

Un guide pour apprendre les notions pour la première fois, comprendre le code et savoir expliquer chaque choix.



Dans ce guide

- Des notions expliquées avec des exemples simples
- Le scanner du TP1 et l'ajout des nombres réels
- La grammaire MICRO et les fonctions du TP2
- Des traces complètes, les erreurs réelles et leurs raisons
- Des exercices corrigés et les deux codes C en annexe

ILISI - Langage MICRO
Fondé sur le support de cours, les six photos et les fichiers C actuels.
12 octobre 2026

Sommaire

Les numéros à gauche désignent les sections ; les numéros à droite indiquent les pages du présent guide. Les entrées sont cliquables dans le PDF.

Avant de commencer : comment lire ce guide 4
1. Compiler : traduire un programme, étape par étape 5
2. Les trois analyses : mots, ordre et sens 6
3. Le petit langage MICRO de nos TP 7
4. Caractère, lexème, modèle et token 8
5. Alphabet, mot et langage : la théorie sans difficulté 9
6. Lire une expression régulière avec des exemples 10
7. Un automate : reconnaître avec des états 11
8. Les bases C nécessaires pour comprendre les TP 12
9. TP1 : quel est exactement l'exercice ? 13
10. TP1 : les données conservées pendant la lecture 14
11. Lire un caractère, le rendre et compter les lignes 15
12. Le déroulement général de scanner() 16
13. Reconnaître un identificateur et les mots réservés 17
14. Trace du TP1 : suivre note_2 + 7 18
15. Reconnaître les entiers : une suite non vide de chiffres 19
16. Ajouter les réels : comprendre le choix du modèle 20
17. Réels : lire la branche C ligne par ligne 21
18. Trace complète : reconnaître 10.32 - 2 22
19. Ponctuation et affectation : un ou deux caractères 23
20. Le moins et les commentaires : pourquoi lire après - ? 24
21. Les erreurs lexicales et la fin du fichier 25
22. TP1 : main(), fichiers et résultat écrit 26
23. Préparer le TP2 : qu'est-ce qu'une grammaire ? 27
24. Lire la grammaire complète de MICRO 28
25. Transformer une règle en fonction C 29
26. Le token courant : regarder sans consommer 30
27. TP2 : son scanner lit les tokens, pas le texte MICRO 31
28. match() : vérifier, puis avancer exactement une fois 32
29. system_goal() et program() : vérifier tout le fichier 33
30. inst_list() : reconnaître une ou plusieurs instructions 34
31. inst() : reconnaître une affectation 35
32. READ et WRITE : deux recettes qui ne demandent pas la même liste 36
33. id_list() et expr_list() : le mécanisme des virgules 37
34. expression() et add_op() : un opérateur exige un élément après lui 38
35. prim() : les éléments simples et les parenthèses 39
36. La récursion expliquée avec des parenthèses 40

37. Un programme complet : du texte aux 20 tokens 41
38. Trace du TP2 : vérifier ce programme complet 42
39. Votre exemple du TP2 : comprendre la première erreur 43
40. Comprendre un diagnostic au lieu de deviner la correction 44
41. Exécuter les deux étapes correctement sous PowerShell 45
42. Les essais réalisés et ce qu'ils vérifient 46
43. Relier chaque notion du cours à ce que nous avons fait 47
44. Les différences, corrections et limites à connaître 48
45. Exercices : vérifier que les idées sont comprises 49
46. Corrections des exercices, expliquées 50
47. Fiche de révision : savoir expliquer les deux TP 51
Références et lecture des annexes 52
Annexe A - Code complet du TP1 53
Annexe B - Code complet du TP2 59

Avant de commencer : comment lire ce guide

Ce document part de zéro. Vous n'avez pas besoin de connaître les mots « lexème », « automate » ou « analyse descendante ». Nous allons d'abord leur donner un sens simple, puis regarder comment les programmes C utilisent ces idées. Le but est de pouvoir expliquer chaque étape, et pas seulement recopier du code.

La lecture suit trois étapes : **comprendre la partie du cours utile**, **comprendre le TP1**, puis **comprendre le TP2 et son lien avec le TP1**. Des traces montrent ce qui se passe dans la mémoire et dans le fichier, caractère après caractère ou token après token.

Les documents utilisés

Le support de cours fourni comporte 107 pages. Les pages 17 à 22 présentent les phases d'analyse ; les pages 38 à 40 présentent les unités lexicales ; les pages 41 à 85 donnent les bases des langages, des expressions régulières et des automates ; les pages 100 à 107 relient ces notions à une implantation.

Les deux photos du TP1 correspondent aux pages imprimées **27 et 28**. Les quatre photos du TP2 correspondent aux pages imprimées **32 à 35**. Ces numéros appartiennent aux feuilles de TP : la page 32 du TP n'est pas la page 32 du PDF de cours.

J'ai également étudié les fichiers actuels **TP1/tp1.c** et **TP2/tp2.c** de votre dossier. Ce sont eux qui permettent de parler précisément de « ce qu'on a fait ». Les anciens PDF d'explication comportent des passages qui ne correspondent plus aux fichiers actuels ; les explications présentes sont fondées sur le code actuel.

Trois repères à garder

- **Sur la feuille** : ce qui est visible dans les photos.
- **Dans votre version** : ce que fait réellement le fichier C actuel.
- **Pour comprendre** : un exemple ou un complément pédagogique ajouté ici.

Le cours fourni développe surtout l'analyse lexicale. La grammaire MICRO et les fonctions syntaxiques détaillées viennent des feuilles du TP2. Leur explication est un complément construit à partir de ces feuilles ; elle n'est pas présentée comme un chapitre absent du PDF de cours.

1. Compiler : traduire un programme, étape par étape

Une idée très simple

Vous écrivez des mots et des symboles qu'un humain peut comprendre. Le processeur, lui, exécute des instructions machine. Un compilateur est un programme qui analyse votre texte et le traduit vers un langage cible. Le cours présente cette idée aux pages 3 à 5.

Mais une traduction correcte commence par une vérification. Imaginez un professeur qui corrige une phrase : il reconnaît d'abord les mots, vérifie ensuite leur ordre, puis vérifie le sens. Un compilateur procède aussi par étapes.

```
Texte source
    -> Analyse lexicale : reconnaître les unités
    -> Analyse syntaxique : vérifier leur organisation
    -> Analyse sémantique : vérifier le sens et les types
    -> Génération de code : produire une traduction
```

Dans nos TP, nous travaillons surtout sur les **deux premières étapes**. Le TP1 prépare les unités lexicales. Le TP2 vérifie leur ordre selon les règles du langage MICRO. Ils ne produisent pas encore un programme machine.

Deux compilations à ne pas confondre

Première compilation : GCC compile `tp1.c` et `tp2.c`. Ce sont des programmes écrits en C. On obtient des exécutables que Windows peut lancer.

Deuxième travail : ces exécutables analysent un fichier écrit en MICRO, par exemple `BEGIN WRITE(2); END`. Ils appliquent les règles de MICRO que nous avons programmées en C.

Une faute dans `tp2.c` peut empêcher GCC de construire l'outil. Une faute dans le fichier MICRO peut être détectée par l'outil déjà construit. Ce sont deux moments différents.

Pourquoi apprendre cela ?

Vous apprenez à transformer une règle abstraite en un programme concret. « Un entier est une suite non vide de chiffres » devient une boucle. « Une instruction finit par un point-virgule » devient un contrôle. Ce passage du cours au code constitue le cœur des deux TP.

Lien avec le cours : pages 3 à 5 et 17. **À retenir :** un analyseur lexical ou syntaxique est une partie d'un compilateur, pas forcément un compilateur complet.

2. Les trois analyses : mots, ordre et sens

Prenons une instruction MICRO :

```
resultat := nombre + 10.32 - 2;
```

L'analyse lexicale voit une suite de caractères. Elle regroupe `resultat`, `:=`, `nombre`, `+`, `10.32`, `-`, `2` et `;`. Elle attribue une catégorie à chaque groupe : identificateur, affectation, réel, etc.

L'analyse syntaxique reçoit ces catégories. Elle demande : « Est-ce bien un identificateur, suivi de :=, suivi d'une expression, suivi de ; ? » Puis elle vérifie l'intérieur de l'expression.

L'analyse sémantique, dans un compilateur plus complet, poserait d'autres questions : `nombre` existe-t-il ? A-t-il une valeur ? Quels sont les types des opérandes ? Comment mélanger entier et réel ? Nos fichiers actuels ne réalisent pas ces contrôles.

Exemple dans un programme	Ce qui pose problème	Étape concernée ici
`x := 2 @ 3;`	@ n'est pas un symbole MICRO prévu	TP1, lexical
`x := 2 + ;`	Il manque un élément après +	TP2, syntaxique
`x := y + 1;`	y pourrait ne pas être initialisé	Contrôle de sens non réalisé

Une entrée peut réussir le TP1 et échouer au TP2. Dans `x := 2 + ;`, tous les caractères appartiennent à des unités connues. Le problème est leur assemblage. Le scanner ne peut donc pas tout vérifier.

Attention au message « programme valide »

Quand le TP2 affiche son succès, il signifie : **la suite de tokens respecte la grammaire programmée**. Il ne signifie pas que le programme a été exécuté, que les variables ont des valeurs, ou que le résultat du calcul est correct.

Le TP2 ne demande aucun nombre à l'utilisateur quand il voit `READ(x);`. Il reconnaît simplement la structure de cette instruction. De même, il ne calcule pas `2 + 3` et n'affiche pas `5` en reconnaissant `WRITE(2 + 3);`.

Lien avec le cours : pages 18 à 22 et 31. **Question de contrôle :** une addition sans deuxième nombre est-elle forcément une erreur lexicale ? Non : les unités peuvent être reconnues, mais leur ordre est incorrect.

3. Le petit langage MICRO de nos TP

MICRO est ici un langage volontairement réduit. Il sert à comprendre l'analyse sans devoir traiter toutes les constructions du C : fonctions, tableaux, types, boucles, chaînes, etc.

```
BEGIN
READ(nombre);
resultat := nombre + 10.32 - 2;
WRITE(resultat);
END
```

Dans le sens habituel de ce langage, `BEGIN` commence le programme et `END` le termine. `READ(nombre);` demande une valeur à placer dans `nombre`. L'affectation `resultat := ...;` donne une valeur à `resultat`. `WRITE(resultat);` affiche une valeur. **Nos outils vérifient ces écritures ; ils ne les exécutent pas.**

Ce que la version actuelle autorise

- Au moins une instruction entre `BEGIN` et `END`.
- Trois types d'instructions : affectation, `READ` et `WRITE`.
- Des identificateurs commençant par une lettre, puis des lettres, chiffres ou `_`.
- Des entiers et des réels décimaux : `5.2`, `.5`, `5.`.
- Les opérateurs binaires `+` et `-`, ainsi que les parenthèses.
- Des commentaires commençant par `--` et finissant à la fin de la ligne ou du fichier.

Les mots réservés doivent être écrits **en majuscules** dans les fichiers actuels. `BEGIN` est reconnu comme début de programme ; `begin` est reconnu comme identificateur. Les exemples de règles sur les feuilles utilisent parfois les mots `begin` et `end` en minuscules comme notation ; il faut suivre la convention du scanner que l'on utilise réellement.

La multiplication, la division, les exposants et le signe unaire ne sont pas prévus. `x := -2.5;` est donc refusé par le TP2 actuel, alors que `x := 0 - 2.5;` est accepté. Le signe `-` est un opérateur séparé, pas une partie du token numérique.

Lien avec le cours : les modèles lexicaux sont décrits aux pages 39 et 40. **Lien avec les TP :** les symboles viennent du scanner des pages 27-28 ; les formes d'instructions viennent des pages 32-35.

4. Caractère, lexème, modèle et token

Quatre mots, quatre rôles

Un **caractère** est un élément du texte : `n`, `2`, `+`, un espace, etc. Un **lexème** est le morceau exact du texte reconnu : `nombre`, `10.32`, `:=`.

Un **modèle** est une règle qui décrit quels morceaux sont autorisés. Exemple : « une lettre, puis zéro ou plusieurs lettres, chiffres ou underscores ». Les lexèmes `x`, `nombre` et `note_2` respectent ce modèle.

Un **token** représente une unité reconnue, généralement par sa catégorie et, si nécessaire, des informations associées. Dans nos programmes C, la valeur retournée est une catégorie de l'énumération `token`. Le lexème et la ligne sont conservés séparément.

Texte exact : lexème	Catégorie retournée	Information associée
`nombre`	`ID`	Le nom précis est `nombre`
`resultat`	`ID`	Le nom précis est `resultat`
`10.32`	`REALLITERAL`	Le texte du nombre est `10.32`
`:=`	`ASSIGNOP`	Opérateur d'affectation
`BEGIN`	`BEGIN_TOKEN`	Mot réservé de début

`nombre` et `resultat` sont différents comme lexèmes, mais ils ont la même catégorie : `ID`. L'analyseur syntaxique peut ainsi utiliser une seule règle d'affectation pour tous les noms de variables.

Pourquoi garder le lexème ?

La catégorie dit **quel genre d'élément** on a trouvé. Le lexème dit **lequel exactement**. Si l'on voulait ensuite exécuter `x := 12;`, il faudrait connaître le nom `x` et la valeur écrite `12`, pas seulement les catégories `ID` et `INTLITERAL`.

Dans le TP1, `token_buffer` conserve le texte reconnu. `token_line` conserve sa ligne d'origine. Le fichier intermédiaire écrit ces informations dans trois colonnes : ligne, catégorie, lexème.

Lien avec le cours : pages 39-40. **À retenir avec une image mentale :** la catégorie est l'étiquette d'une boîte ; le lexème est l'objet précis qu'on met dans cette boîte.

5. Alphabet, mot et langage : la théorie sans difficulté

Un **alphabet** est un ensemble de symboles autorisés. Si l'alphabet est `{0, 1}`, on peut construire `0`, `101`, `0011`, etc. Un **mot** est une suite finie de symboles de cet alphabet. En théorie, « mot » ne veut donc pas dire seulement un mot français.

Un **langage** est un ensemble de mots. Par exemple, le langage des suites de chiffres non vides contient `0`, `7`, `25`, `003`, etc. Il ne contient pas `a3`, car une lettre n'est pas un chiffre.

Le mot vide

Le mot vide, noté **epsilon**, ne contient aucun symbole. Sa longueur vaut zéro. Il est utile pour exprimer « il est possible de ne rien ajouter ».

Ne confondez pas **le langage vide**, qui ne contient aucun mot, et **le langage contenant le mot vide**, qui contient un élément : le mot de longueur zéro. Une boîte vide et une boîte contenant une feuille blanche ne représentent pas la même chose.

Coller des morceaux : la concaténation

Concaténer signifie mettre les morceaux l'un après l'autre. Concaténer `ab` et `12` donne `ab12`. Ce n'est pas une addition arithmétique. L'ordre compte : `ab12` n'est pas `12ab`.

Si `L` est l'ensemble des lettres et `D` l'ensemble des chiffres, le modèle d'un identificateur MICRO peut s'écrire :

```
ID = L (L | D | _)*
```

On choisit d'abord une lettre. Puis on ajoute autant de lettres, chiffres ou underscores que l'on veut, éventuellement aucun. C'est pourquoi `x` est déjà un identificateur valide.

Le changement d'échelle est important : le TP1 reconnaît des mots **sur un alphabet de caractères**. Le TP2 reconnaît des programmes **sur un vocabulaire de tokens**. On travaille d'abord avec `R`, `E`, `A`, `D`, puis avec le token unique `READ_TOKEN`.

Lien avec le cours : pages 41, 43, 45 à 55. Cette théorie sert directement à préciser ce que vos boucles doivent accepter.

6. Lire une expression régulière avec des exemples

Une expression régulière décrit un ensemble de textes possibles. Ce n'est pas encore le code du scanner : c'est la **règle à respecter**. Un automate ou un programme peut ensuite reconnaître cette règle.

Notation utilisée ici	Sens simple	Exemple
`A B`	Un morceau A, puis un morceau B	Lettre puis chiffre
`A | B`	A ou B	Lettre ou chiffre
`A*`	Zéro, une ou plusieurs répétitions	Aucune lettre est possible
`A+`	Une ou plusieurs répétitions	Au moins un chiffre
`A?`	Zéro ou une occurrence	Partie facultative

Attention : dans certaines pages du cours, `a+b` signifie **a ou b**. Dans les abréviations usuelles comme `chiffre+`, le plus placé après un élément signifie **une ou plusieurs répétitions**. Et dans un programme MICRO, `+` signifie l'addition. Il faut toujours regarder le contexte.

```
L  = [A-Za-z]
D  = [0-9]
ID = L (L | D | _)*
ENTIER = D+
REEL = D+ '.' D* | '.' D+
```

Le point entre guillemets est le **caractère point**, pas le méta-caractère d'un outil de regex. Dans une regex logicielle, on l'écrirait souvent `\.` pour demander un point réel.

Lire le réel à voix haute : soit un ou plusieurs chiffres, un point, puis zéro ou plusieurs chiffres ; soit un point, puis au moins un chiffre. On retrouve ainsi `5.2`, `5.` et `.5`. Le point seul ne suffit pas.

Pourquoi préciser le modèle avant le code ?

Si l'on écrit simplement « accepter les réels », plusieurs questions restent ouvertes : `.5` est-il accepté ? Et `5.` ? Et `1e3` ? Le code ne peut pas deviner la réponse. La version actuelle choisit les deux formes décimales ci-dessus, sans exposant.

Lien avec le cours : pages 64-67 et 76-78. Le modèle des nombres du cours est plus riche ; votre TP implémente une partie de ces formes.

7. Un automate : reconnaître avec des états

Un **état** représente où l'on en est dans la reconnaissance. Une **transition** dit comment changer d'état quand on lit un caractère. Un **état acceptant** indique qu'un préfixe déjà lu constitue une unité autorisée.

Imaginez la lecture d'un nombre décimal. Avant toute lecture, on ne sait rien. Après `5`, on a déjà un entier. Après `5.`, la version actuelle accepte déjà un réel. Après `5.2`, on a un réel avec une partie après le point.

État	Chiffre lu	Point lu	Autre caractère ou fin
Départ	Aller à Entier	Aller à Point initial	Aucun nombre reconnu
Entier, acceptant	Rester en Entier	Aller à Décimal	Terminer l'entier
Point initial, non acceptant	Aller à Décimal	Échec pour le point initial	Échec pour le point initial
Décimal, acceptant	Rester en Décimal	Arrêter ce nombre	Terminer le réel

Exemple : `.5;`

On part de Départ. Le point mène à Point initial, qui n'est pas acceptant : on n'a encore aucun chiffre. Le `5` mène à Décimal, acceptant. Le `;` ne prolonge pas le nombre : on termine `.5`, puis on laisse `;` au prochain appel.

Exemple : `5.2.3`

Le premier nombre reconnu est `5.2`. Le deuxième point ne peut pas prolonger ce nombre. Dans votre code, il est remis dans le flux, puis un nouvel appel reconnaît `.3`. Le TP1 produit donc **deux réels**, et le TP2 refuse cette succession dans une expression sans opérateur.

Un automate fini garde une quantité limitée d'information : l'état actuel suffit. Dans votre scanner, les états ne sont pas tous nommés explicitement. Ils apparaissent sous forme de branches, de boucles et du drapeau `is_real`. Le comportement est néanmoins comparable à celui d'un automate.

Lien avec le cours : pages 79, 84-85 et 100-103. **À retenir :** on n'essaie pas au hasard ; chaque caractère détermine une action selon ce qui a déjà été lu.

8. Les bases C nécessaires pour comprendre les TP

Une fonction renvoie une catégorie

```
typedef enum { ID, INTLITERAL, REALLITERAL, SCANEOF } token;

token scanner(void);
```

Une énumération crée des noms pour des valeurs entières. `token` devient le nom du type. `scanner(void)` n'a pas de paramètre et renvoie une valeur de ce type. Il ne renvoie pas toute la chaîne : celle-ci reste dans `token_buffer`.

`return ID;` termine immédiatement **l'appel de la fonction**. Ce n'est pas la fin de l'analyse de tout le fichier. Le programme appelant pourra rappeler `scanner()` pour obtenir l'unité suivante.

Les symboles C qui reviennent souvent

- `=` affecte une valeur : `c = read_char()` lit puis range le résultat dans `c`.
- `==` compare : `c == '.'` demande si `c` est un point.
- `!=` signifie « différent de » ; `&&` signifie « et » ; `||` signifie « ou ».
- `continue` passe directement au tour suivant d'une boucle.
- `break` sort de la boucle ou du `switch` le plus proche ; il ne termine pas forcément la fonction.
- `return` quitte la fonction ; `exit(EXIT_FAILURE)` termine tout le programme en échec.

Pourquoi un caractère lu est stocké dans un int ?

`getchar()` et `fgetc()` renvoient un caractère représenté comme un entier, ou la valeur spéciale `EOF`. `EOF` signifie « plus de caractère disponible ». Utiliser `int` permet de conserver ce résultat complet avant de savoir si c'est un caractère ou la fin.

Une chaîne C finit par le caractère `\0`. Pour garder `abc`, il faut quatre emplacements : `a`, `b`, `c` et `\0`. Ce dernier caractère n'est pas la lettre zéro et n'est pas le marqueur `EOF`.

Dans ces fichiers, `static` au niveau du fichier limite la visibilité au fichier C concerné. Une variable globale reste disponible entre les appels. `const char *` désigne ici un accès à du texte qu'on ne modifie pas par ce pointeur.

Lien cours-TP : ce sont des outils C utilisés pour réaliser les règles lexicales et grammaticales, pas des règles du langage MICRO.

9. TP1 : quel est exactement l'exercice ?

Sur la feuille, la fonction `scanner()` reconnaît les identificateurs, les mots réservés, les entiers, certains symboles et les commentaires. L'exercice 1.1 demande d'enrichir MICRO pour qu'il traite aussi des réels, par exemple `5.2`, et de préciser les changements nécessaires dans `scanner()`.

Ce qui manque au scanner de départ

```
Texte : 5.2
Ancienne branche : lire tous les chiffres
Résultat initial : INTLITERAL pour 5
Puis : le point n'a aucune branche prévue
```

L'ancien code s'arrête dès qu'il rencontre un caractère qui n'est pas un chiffre. Il sait donc reconnaître `5`, mais pas tout `5.2`. Le changement doit se faire dans la reconnaissance des nombres, pas dans la reconnaissance des identificateurs.

Les décisions à prendre

1. Choisir le modèle. Votre version accepte `5.2`, `5.` et `.5`. La feuille donne `5.2` comme exemple, mais ne tranche pas explicitement toutes les variantes. Le support de cours montre des formes décimales aux pages 40, 78 et 103. L'acceptation des variantes relève ici du choix d'implémentation.

2. Ajouter une catégorie. Il faut distinguer un entier d'un réel : votre code ajoute `REALLITERAL` dans l'énumération et dans les noms affichés.

3. Adapter la lecture. Il faut lire un éventuel point et les chiffres qui le suivent. La fonction doit conserver le nombre entier ou décimal complet dans le tampon.

4. Préserver le caractère suivant. Si l'on lit `5.2;`, le `;` doit encore être disponible pour le prochain token.

5. Préparer le TP2. Le scanner peut reconnaître les réels, mais le parser doit aussi les autoriser dans les expressions. Cet ajout se fait dans `prim()`.

Résultat attendu du TP1 : une liste de catégories et de lexèmes. Reconnaître un réel ne veut pas encore dire le convertir en valeur flottante ou effectuer des calculs.

10. TP1 : les données conservées pendant la lecture

Votre code utilise un fichier source, un tampon pour le lexème, des numéros de ligne et un compteur d'erreurs. Chaque donnée répond à un besoin concret.

Nom	Rôle	Pourquoi on en a besoin
`source_file`	Fichier MICRO ouvert	Lire les caractères
`source_name`	Nom du fichier	Donner des erreurs compréhensibles
`token_buffer[256]`	Texte de l'unité actuelle	Écrire le lexème en sortie
`token_buffer_index`	Position du prochain ajout	Savoir où ranger le caractère
`current_line`	Ligne atteinte dans la lecture	Suivre les retours à la ligne
`token_line`	Ligne de début de l'unité	Associer la bonne ligne au token
`error_count`	Nombre d'erreurs lexicales	Signaler l'échec final

```
static void clear_buffer(void)
{
    token_buffer_index = 0;
    token_buffer[0] = '\0';
}
```

On remet l'indice à zéro : le prochain caractère sera écrit au début. On place immédiatement `\0` : le tampon représente maintenant une chaîne vide. Il n'est pas nécessaire d'effacer tous les anciens octets, car les fonctions de chaîne s'arrêtent au premier `\0`.

```
token_buffer[token_buffer_index++] = (char)c;
token_buffer[token_buffer_index] = '\0';
```

La première ligne range `c` à la position actuelle, puis avance l'indice. La deuxième maintient la fin de chaîne. Après `a`, puis `b`, le contenu utile passe de `"a"` à `"ab"`.

`buffer_char()` vérifie aussi la place disponible **avant** l'ajout. Dans 256 cases, on peut garder au maximum 255 caractères utiles et la terminaison `\0`. Sans cette vérification, un lexème trop long pourrait écrire hors du tableau.

Lien avec le cours : les pages 38 et 40 parlent de ligne et de lexème. Le tableau de caractères et son indice sont notre manière de conserver ces informations en C.

11. Lire un caractère, le rendre et compter les lignes

Sur la feuille, la lecture se fait par `getchar()` depuis `stdin`. Votre version lit un fichier avec `fgetc(source_file)`, encapsulé dans `read_char()`.

```
static int read_char(void)
{
    int c = fgetc(source_file);
    if (c == '\n')
        ++current_line;
    return c;
}
```

Quand on lit un retour à la ligne, le prochain caractère appartient à la ligne suivante : on augmente `current_line`. Les autres caractères ne changent pas le numéro de ligne.

```
static void unread_char(int c)
{
    if (c == EOF)
        return;
    if (c == '\n')
        --current_line;
    ungetc(c, source_file);
}
```

Rendre un caractère signifie demander que la prochaine lecture retrouve ce caractère. On n'efface pas le fichier et on ne déplace pas son contenu. `ungetc` remet un caractère dans le flux de lecture.

Pourquoi revenir d'un caractère ?

Pour reconnaître `abc;`, on lit `a`, `b`, `c`, puis `;`. Ce dernier caractère prouve que le nom est fini, mais il ne fait pas partie du nom. On remet donc `;`, puis on renvoie `ID`. Au prochain appel, le scanner lit `;` et renvoie `SEMICOLON`.

Si le caractère rendu est `\n`, il faut annuler l'incrément fait par `read_char()`. Sinon, le même retour à la ligne serait compté deux fois : une fois avant de le rendre et une fois quand on le relira.

La valeur `EOF` n'est pas un caractère à rendre. Votre fonction retourne immédiatement dans ce cas. La feuille appelle directement `ungetc(c, stdin)` ; la version actuelle rend ce comportement plus explicite.

Lien avec le cours : page 106, l'opération `retract(1)` joue le rôle d'un retour sur le caractère de séparation. Ici, `unread_char()` réalise cette idée.

12. Le déroulement général de scanner()

Il faut comprendre une idée centrale : **un appel à `scanner()` renvoie au plus un token**. Il peut lire plusieurs caractères, ignorer des espaces et sauter un commentaire, mais dès qu'il trouve une unité à renvoyer, il exécute un `return`.

```
while ((in_char = read_char()) != EOF) {
    if (isspace((unsigned char)in_char))
        continue;

    clear_buffer();
    token_line = current_line;
    /* Reconnaître l'unité qui commence ici. */
}
clear_buffer();
token_line = current_line;
return SCANEOF;
```

Lire ces lignes lentement

La condition appelle d'abord `read_char()`, range le résultat dans `in_char`, puis compare ce résultat à `EOF`. S'il reste un caractère, on entre dans la boucle.

`isspace()` reconnaît les blancs : espaces, tabulations, retours à la ligne, etc. `continue` les ignore et relance la lecture. Le scanner ne fabrique donc pas de token « espace ».

Quand on trouve un caractère utile, on vide le tampon et on mémorise la ligne de début. Ensuite, on choisit une branche : lettre, chiffre ou point, ponctuation, opérateur, commentaire ou erreur.

À la fin du fichier, on vide le tampon et on renvoie `SCANEOF`. **EOF** est le résultat spécial d'une lecture de caractères ; **SCANEOF** est la catégorie spéciale destinée au code qui demande des tokens.

Pourquoi les conversions vers unsigned char ?

Les fonctions de `<ctype.h>` comme `isspace`, `isalpha` et `isdigit` attendent `EOF` ou une valeur représentable en `unsigned char`. Pour un caractère lu, la conversion protège l'appel contre une valeur signée négative. La fin est testée avant les boucles qui examinent ces caractères.

Lien avec le cours : pages 18, 38 et 100. Lire linéairement, supprimer les blancs, produire une unité, puis recommencer : c'est exactement le rôle de l'analyse lexicale.

13. Reconnaître un identificateur et les mots réservés

```
if (isalpha((unsigned char)in_char)) {
    buffer_char(in_char);
    for (c = read_char();
         c != EOF && (isalnum((unsigned char)c) || c == '_');
         c = read_char())
        buffer_char(c);
    unread_char(c);
    return check_reserved();
}
```

Le premier caractère doit être une lettre. On le range immédiatement. La boucle lit ensuite les caractères autorisés à l'intérieur du nom : lettre, chiffre ou underscore. `isalnum` réunit les tests « lettre » et « chiffre ».

Pourquoi le premier caractère a-t-il un test différent ? Parce que la règle MICRO ne permet pas de commencer par un chiffre ou _. `note_2` est un identificateur ; `_note` n'en est pas un dans cette version. Le cours présente aussi des identificateurs C acceptant _ au début : ce n'est pas le choix des TP actuels.

Pourquoi check_reserved() vient après la lecture du nom complet ?

`BEGIN` respecte aussi la forme générale d'un identificateur. Il faut donc reconnaître le texte entier, puis déterminer s'il est un mot spécial.

```
if (strcmp(token_buffer, "BEGIN") == 0)
    return BEGIN_TOKEN;
/* Même principe pour END, READ et WRITE. */
return ID;
```

`strcmp(a, b)` renvoie zéro quand les deux chaînes sont égales. Ce n'est pas un test par `==` entre tableaux de caractères : on compare leur contenu.

`BEGINNER` doit rester un seul `ID`. Si l'on renvoyait `BEGIN_TOKEN` dès les cinq premières lettres, on couperait incorrectement `BEGINNER` en deux morceaux. C'est pourquoi on lit d'abord le lexème complet.

Les comparaisons de votre version sont sensibles à la casse. `READ` est réservé ; `read` est un identificateur.

Lien avec le cours : pages 39-40, 76-78 et 102. **À retenir :** la forme du mot est reconnue d'abord ; sa place dans la liste des mots réservés est vérifiée ensuite.

14. Trace du TP1 : suivre note_2 + 7

Imaginons le texte `note_2 + 7`. On appelle `scanner()` pour la première fois. Le tampon est prêt à conserver une seule unité.

Caractère lu	Décision	Tampon après la décision
`n`	Lettre initiale : commencer un nom	`n`
`o`	Lettre autorisée : ajouter	`no`
`t`	Lettre autorisée : ajouter	`not`
`e`	Lettre autorisée : ajouter	`note`
`_`	Underscore autorisé : ajouter	`note_`
`2`	Chiffre autorisé après la lettre : ajouter	`note_2`
espace	Ne prolonge pas le nom : rendre le caractère	`note_2`

`check_reserved()` compare `note_2` aux quatre mots réservés. Aucun ne correspond. Le premier appel renvoie donc **ID**, avec le lexème **note_2**.

Deuxième appel

Il relit l'espace rendu, l'ignore, puis lit `+`. Ce caractère suffit pour reconnaître l'opérateur d'addition. Le tampon est `+`, et la catégorie retournée est `PLUSOP`.

Troisième appel

Il ignore l'espace suivant, puis lit `7`. La branche numérique reconnaît un entier. Le lexème est `7`, et la catégorie retournée est `INTLITERAL`. Si le fichier finit immédiatement, il n'y a aucun caractère à rendre.

Quatrième appel

Il ne trouve plus de caractère. Il renvoie `SCANEOF`. Le programme appelant sait alors qu'il peut arrêter de demander des unités.

```
Texte               note_2 + 7
Catégories          ID PLUSOP INTLITERAL SCANEOF
Lexèmes conservés   note_2    +    7
```

L'espace a servi à séparer les morceaux, mais n'a pas été écrit dans le résultat. Le même découpage serait obtenu pour `note_2+7`. En revanche, `note_27` formerait un seul identificateur : sans séparateur ou symbole différent, tous ces caractères prolongent le même nom.

Petit exercice : `READx` donne-t-il `READ` puis `ID` ? Non. Il donne un seul `ID`, car le nom complet n'est pas exactement le mot réservé `READ`.

15. Reconnaître les entiers : une suite non vide de chiffres

Dans le scanner initial de la feuille, la branche entière suit cette logique : si le premier caractère est un chiffre, le conserver, puis lire et conserver tous les chiffres suivants.

```
buffer_char(in_char);
for (c = read_char(); c != EOF && isdigit((unsigned char)c);
     c = read_char())
    buffer_char(c);
unread_char(c);
return INTLITERAL;
```

Cet extrait exprime la logique de l'entier avec une vérification explicite d'EOF. Dans votre version étendue, elle fait partie d'une branche numérique plus grande, qui peut aussi rencontrer un point.

Exemple : `203;`

`2` déclenche la branche. `0` et `3` sont ajoutés. `;` ne correspond pas à un chiffre : on le rend. Le token retourné est `INTLITERAL` avec le lexème `203`.

Pourquoi le modèle est-il `D+` et pas `D*` ? Parce qu'un nombre entier écrit doit contenir au moins un chiffre. Un texte vide n'est pas un entier. Le premier chiffre lu avant la boucle garantit cette présence obligatoire.

Une écriture et une valeur, ce n'est pas la même chose

Le scanner conserve `203` comme chaîne de caractères. Il ne fait pas automatiquement `atoi`, `strtol` ou `strtod`. Il classe le morceau ; il ne calcule pas sa valeur dans nos TP.

`007` est également reconnu comme `INTLITERAL`. Le scanner ne décide pas ici d'une convention de base octale ou d'une valeur numérique particulière. Il applique uniquement le modèle « chiffres ».

Et `12abc` ?

Dans votre version, la lecture entière s'arrête au `a`. On obtient `INTLITERAL("12")`, puis `ID("abc")`. Le TP1 réussit ce découpage. Dans `x := 12abc;`, le TP2 attend ensuite un point-virgule après `12` et rencontre `abc` : il refuse la syntaxe.

Lien avec le cours : pages 78, 103 et 107. Une erreur apparente dans un nombre n'est pas toujours détectée au niveau lexical : cela dépend des règles de découpage effectivement programmées.

16. Ajouter les réels : comprendre le choix du modèle

Votre version utilise deux formes de réels :

```
Chiffres, puis point, puis éventuellement des chiffres : 5.2 ou 5.
Point, puis au moins un chiffre : .5
```

La branche numérique commence donc lorsque `in_char` est un chiffre **ou** un point. Le drapeau `is_real` indique si l'on a rencontré le point qui fait appartenir l'unité à la catégorie réelle.

```
if (isdigit((unsigned char)in_char) || in_char == '.') {
    int is_real = in_char == '.';
    buffer_char(in_char);
    /* Lire les chiffres et éventuellement un point. */
    return is_real ? REALLITERAL : INTLITERAL;
}
```

`int is_real = in_char == '.';` range le résultat de la comparaison : 1 si l'on commence par un point, 0 sinon. On utilise ici un entier comme indicateur vrai/faux.

L'opérateur `? :` est un choix compact : si `is_real` est vrai, renvoyer `REALLITERAL` ; sinon renvoyer `INTLITERAL`. Il ne convertit pas le texte en un nombre réel.

Texte	Catégorie de la version actuelle	Justification
`5`	`INTLITERAL`	Aucun point
`5.2`	`REALLITERAL`	Chiffres et point
`5.`	`REALLITERAL`	Chiffres avant le point ; suite facultative
`.5`	`REALLITERAL`	Au moins un chiffre après le point initial
`.`	Erreur lexicale	Aucun chiffre
`1e3`	Entier `1`, puis identificateur `e3`	Aucun exposant prévu

La feuille demande les réels, mais ne demande pas explicitement la notation scientifique. Il serait possible de l'ajouter ; il faudrait alors des états et des contrôles supplémentaires pour `e`, le signe de l'exposant et ses chiffres obligatoires.

Lien avec le cours : pages 40, 78 et 103. **Lien avec le TP2 :** `prim()` doit accepter `REALLITERAL`, sinon cette nouvelle catégorie sera reconnue puis refusée.

17. Réels : lire la branche C ligne par ligne

```
if (in_char == '.') {
    c = read_char();
    if (c == EOF || !isdigit((unsigned char)c)) {
        unread_char(c);
        lexical_error(in_char);
        continue;
    }
    buffer_char(c);
}
```

Si le nombre commence par un point, on lit immédiatement un caractère supplémentaire. Il doit être un chiffre. Sans cela, le point ne constitue pas une unité numérique autorisée. On rend le caractère supplémentaire, on signale l'erreur pour le point, puis on reprend la recherche d'une unité.

Si l'on a déjà lu `.5`, le `5` est rangé. La boucle suivante commence donc au caractère **après** ce `5`. Elle n'oublie pas et ne relit pas ce premier chiffre.

```
for (c = read_char();
     c != EOF && isdigit((unsigned char)c);
     c = read_char())
    buffer_char(c);
```

Cette boucle lit les chiffres. Pour `10.32`, elle ajoute d'abord le `0` après le premier `1`, puis s'arrête sur le point. Pour `.532`, elle ajoute `3` et `2` après `.5`.

```
if (!is_real && c == '.') {
    is_real = 1;
    buffer_char(c);
    for (c = read_char();
         c != EOF && isdigit((unsigned char)c);
         c = read_char())
        buffer_char(c);
}
unread_char(c);
return is_real ? REALLITERAL : INTLITERAL;
```

On ajoute un point seulement si le nombre n'a pas déjà commencé par un point. Le drapeau devient vrai. Les chiffres après ce point sont lus. Leur absence est autorisée dans cette branche : c'est pourquoi `5.` est accepté. Enfin, le caractère qui a arrêté la lecture est rendu.

Pourquoi `!is_real` **?** Cela empêche d'ajouter un deuxième point dans le même lexème. La branche accepte un nombre décimal avec un seul point ; elle ne rassemble pas tout `5.2.3`.

18. Trace complète : reconnaître 10.32 - 2

L'entrée est `10.32 - 2;`. Nous suivons un appel à `scanner()` jusqu'au premier `return`.

Moment	Caractère	Action	Tampon	is_real
Entrée dans la branche	`1`	Premier chiffre rangé	`1`	0
Boucle avant le point	`0`	Chiffre ajouté	`10`	0
Fin de cette boucle	`.`	Arrêt de la boucle des chiffres	`10`	0
Test du point	`.`	Ajouter le point ; changer le drapeau	`10.`	1
Boucle après le point	`3`	Chiffre ajouté	`10.3`	1
Même boucle	`2`	Chiffre ajouté	`10.32`	1
Fin de la boucle	espace	Rendre ce caractère	`10.32`	1

Le premier appel renvoie `REALLITERAL`. Il n'a pas reconnu le signe `-` ni le nombre `2`. Ces éléments appartiennent aux appels suivants.

Le deuxième appel ignore l'espace, lit `-`, puis lit un caractère pour vérifier si l'on a `--`. Il trouve un espace, pas un deuxième moins. Il rend cet espace et renvoie `MINUSOP`.

Le troisième appel ignore cet espace et reconnaît `2`. Il lit `;` pour découvrir que l'entier est fini, rend `;` et renvoie `INTLITERAL`. Le quatrième appel renvoie `SEMICOLON`.

```
REALLITERAL("10.32")
MINUSOP("-")
INTLITERAL("2")
SEMICOLON(";")
```

Comparer avec `.5;`

Le point initial met immédiatement `is_real` à 1. Le scanner exige puis ajoute `5`. La boucle lit `;` et s'arrête. Le test `!is_real` est faux : aucun autre point n'est ajouté. On rend `;` et on retourne `REALLITERAL`.

À retenir : lire un caractère pour vérifier la fin d'un lexème n'autorise pas à le perdre. Le caractère rendu est le pont entre deux appels successifs.

19. Ponctuation et affectation : un ou deux caractères

Les symboles `(`, `)`, `;`, `,` et `+` se reconnaissent immédiatement. Un seul caractère suffit. Votre code a déjà ajouté ce caractère au tampon avant son `switch`.

```
switch (in_char) {
    case '(': return LPAREN;
    case ')': return RPAREN;
    case ';': return SEMICOLON;
    case ',': return COMMA;
    case '+': return PLUSOP;
    /* Autres branches ensuite. */
}
```

Pourquoi les noms `LPAREN` et `RPAREN` ? Ils abrègent « left parenthesis » et « right parenthesis », c'est-à-dire parenthèse ouvrante et fermante. `SEMICOLON` signifie point-virgule ; `COMMA` signifie virgule.

Le cas particulier de :=

```
case ':':
    c = read_char();
    if (c == '=') {
        buffer_char(c);
        return ASSIGNOP;
    }
    unread_char(c);
    lexical_error(in_char);
    break;
```

Le `:` seul n'est pas un token MICRO prévu. Il faut vérifier le caractère qui suit. Si c'est `=`, les deux caractères forment un seul opérateur `ASSIGNOP` et le lexème devient `:=`.

Si l'on lit `:x`, le `x` n'est pas perdu : on le rend. On signale le `:` incorrect. Le `break` quitte le `switch`, puis la boucle extérieure continue. Le prochain tour pourra reconnaître `x`.

Trois écritures très différentes

`x := 2;` utilise le bon opérateur. `x = 2;` contient un `=` isolé non prévu : erreur lexicale. `x : = 2;` ne forme pas `:=`, car les deux caractères sont séparés : il y a également des erreurs lexicales.

Lien avec le cours : page 101, des opérateurs comme `<` et `<=` demandent aussi de lire un caractère d'anticipation. Nos feuilles appliquent le même principe à `:=`.

20. Le moins et les commentaires : pourquoi lire après - ?

Quand le scanner rencontre `-`, il ne sait pas encore si c'est une soustraction ou le début d'un commentaire. Il lit donc un caractère supplémentaire.

```
case '-':
    c = read_char();
    if (c == '-') {
        do {
            in_char = read_char();
        } while (in_char != '\n' && in_char != EOF);
    } else {
        unread_char(c);
        return MINUSOP;
    }
    break;
```

Si le caractère supplémentaire n'est pas `-`, on le rend et on renvoie `MINUSOP`. Si c'est `-`, on lit tout le reste de la ligne sans produire de token pour ce texte.

Exemple à suivre

```
x := 5; -- ceci est un commentaire
WRITE(x);
```

Le scanner produit les tokens de `x := 5;`. Lorsqu'il atteint `--`, il ignore le commentaire. Il consomme le retour à la ligne, puis reprend sa recherche. Le prochain token utile est `WRITE_TOKEN`, à la ligne suivante.

Une différence entre la feuille et votre version

Sur la feuille, la boucle du commentaire s'arrête seulement sur `\n`. Si le fichier finit avant un retour à la ligne, `getchar()` continue à renvoyer `EOF` et cette condition peut provoquer une boucle sans fin. Votre version ajoute le test `in_char != EOF` : un commentaire au bout du fichier est correctement terminé.

Attention à 5--2

Dans votre version, `x := 5--2;` signifie « le nombre 5, puis un commentaire ». Le scanner n'émet pas deux soustractions. Le point-virgule est également dans le commentaire ; le TP2 détectera qu'il manque. Pour écrire une soustraction, utilisez `5 - 2`. Les signes unaires ne sont de toute façon pas reconnus par la grammaire actuelle.

Lien avec le cours : pages 18 et 38, élimination des commentaires. **À retenir :** le scanner décide d'abord du découpage ; le parser ne voit jamais le contenu supprimé d'un commentaire.

21. Les erreurs lexicales et la fin du fichier

Quand aucun modèle ne correspond au caractère utile lu, votre scanner appelle `lexical_error()`. La fonction affiche le fichier, la ligne et le caractère incorrect, puis augmente `error_count`.

```
BEGIN WRITE(1@); END
            ^ caractère non prévu
```

Le scanner signale `@`, puis continue. Il peut donc produire un fichier de tokens même si des erreurs lexicales ont été rencontrées. À la fin, `main()` vérifie le compteur et renvoie un code d'échec.

Pourquoi le code de retour est indispensable

Après suppression de `@`, les tokens restants ressemblent à ceux de `WRITE(1);`, qui est syntaxiquement correct. Si l'on ignore l'échec du TP1 et lance quand même le TP2, on pourrait obtenir un succès syntaxique pour une entrée qui avait une erreur lexicale.

La règle pratique est donc : **ne lancer le TP2 que si le TP1 a terminé avec succès**. Le fichier produit n'est pas une preuve que l'entrée était correcte ; il faut aussi vérifier le code de retour.

EOF : ce qu'il faut corriger dans la feuille

La photo montre `if (eof(stdin)) return SCANEOF;`. `eof` n'est pas une fonction standard de la bibliothèque C ; le nom standard est `feof`. Mais remplacer le nom ne suffit pas : `feof()` constate une fin déjà rencontrée par une lecture, il ne prédit pas le résultat de la prochaine lecture.

La méthode la plus claire ici est celle de votre version : lire avec `fgetc`, comparer le résultat à `EOF`, puis renvoyer `SCANEOF` après la boucle. La feuille ne montre pas de `return SCANEOF` à la sortie de cette boucle ; une fonction non `void` doit renvoyer une valeur cohérente sur ce chemin.

Votre version traite aussi la taille du tampon. Une unité de 256 caractères dépasse les 255 caractères utiles autorisés : elle provoque un arrêt immédiat, au lieu d'écraser de la mémoire.

Lien avec le cours : pages 31 et 107. **Limite actuelle :** le TP1 ne contrôle pas explicitement `ferror()` à la sortie de la lecture, contrairement au TP2.

22. TP1 : main(), fichiers et résultat écrit

`main()` organise le travail. Les fonctions du scanner reconnaissent les unités ; `main()` ouvre les fichiers, appelle le scanner autant que nécessaire et écrit le résultat.

L'ordre des opérations

1. Vérifier les arguments : un fichier source est obligatoire, un fichier de sortie est facultatif.
2. Ouvrir la source avec `fopen(..., "r")`. Si l'ouverture échoue, arrêter avec un diagnostic.
3. Ouvrir la sortie avec `fopen(..., "w")`. Ce mode crée le fichier ou remplace son contenu.
4. Écrire les titres des trois colonnes.
5. Répéter l'appel au scanner jusqu'à `SCANEOF`.
6. Fermer les deux fichiers et examiner le compteur d'erreurs.

```
while ((current_token = scanner()) != SCANEOF) {
    fprintf(output_file, "%-8u %-16s %s\n",
            token_line, token_name(current_token), token_buffer);
}
```

Cette boucle appelle le scanner, conserve la catégorie obtenue et teste la fin. Pour un token ordinaire, elle écrit la ligne, le nom de catégorie et le lexème. Les nombres comme `8` et `16` règlent les largeurs de présentation ; ils ne font pas partie de la grammaire MICRO.

```
LIGNE    UNITE           LEXEME
-----    -----           ------
1        BEGIN           BEGIN
2        READ            READ
2        LPAREN          (
2        ID              nombre
2        RPAREN          )
2        SEMICOLON       ;
```

Le token `SCANEOF` n'est pas écrit comme une ligne dans ce fichier. La boucle s'arrête lorsqu'elle le reçoit. Le TP2 recrée son propre marqueur de fin lorsqu'il termine la lecture des lignes de tokens.

`token_name()` transforme les valeurs de l'énumération en noms lisibles. Par exemple, `BEGIN_TOKEN` devient le texte `BEGIN`. C'est ce nom textuel que le lecteur du TP2 devra reconnaître.

Lien avec le cours : pages 38 et 100. Le fichier intermédiaire matérialise l'échange d'unités lexicales entre deux étapes.

23. Préparer le TP2 : qu'est-ce qu'une grammaire ?

Une grammaire donne les règles d'organisation des unités. Le scanner sait qu'un point-virgule est un `SEMICOLON`. La grammaire dit **où ce point-virgule doit apparaître**.

Terminaux et non-terminaux

Un **terminal** est un élément reconnu directement dans l'entrée du parser : `ID`, `BEGIN`, `;`, etc. Un **non-terminal** désigne une construction composée : programme, instruction, expression, liste.

```
<program> ::= BEGIN <inst_list> END
```

On lit : « un programme se compose de BEGIN, puis d'une liste d'instructions, puis de END ». `::=` signifie « se définit par ». Les chevrons désignent ici un nom de construction ; ils ne sont pas des caractères à écrire dans le programme MICRO.

Les deux côtés de la règle

À gauche se trouve ce que l'on veut reconnaître. À droite se trouve sa recette. Pour reconnaître un programme, il faut appliquer la recette dans l'ordre. Ce n'est pas une affectation de variable C.

```
<inst> ::= ID := <expr> ;
```

Le premier `:=` dans cette règle est le vrai opérateur du langage MICRO. La règle demande un identificateur, l'opérateur d'affectation, une expression et un point-virgule. Le nom `<expr>` sera développé par une autre règle.

Deux alphabets successifs

Au TP1, le texte `resultat` est reconnu caractère par caractère. Au TP2, le nom précis peut varier : l'élément important pour la structure est sa catégorie `ID`. La grammaire peut donc reconnaître `x := 1;` et `total := 1;` par la même recette.

Le cours présente l'analyse grammaticale et l'arbre de dérivation aux pages 20-21. Les recettes particulières à MICRO sont visibles dans les commentaires des fonctions sur les feuilles du TP2.

À retenir : une grammaire n'explique pas comment exécuter une instruction ; elle décrit comment une instruction autorisée doit être construite.

24. Lire la grammaire complète de MICRO

Voici les règles reconnues par votre version. L'ajout `REALLITERAL` complète les feuilles grâce à l'extension du TP1.

```
system_goal ::= program SCANEOF
program     ::= BEGIN inst_list END
inst_list   ::= inst { inst }

inst        ::= ID := expr ;
              | READ ( id_list ) ;
              | WRITE ( expr_list ) ;

id_list     ::= ID { , ID }
expr_list   ::= expr { , expr }
expr        ::= prim { add_op prim }
add_op      ::= + | -
prim        ::= ( expr ) | ID | INTLITERAL | REALLITERAL
```

Dans cette notation EBNF, `{ ... }` signifie **répéter zéro ou plusieurs fois**. Les accolades ne sont pas des symboles MICRO à écrire. Le signe | signifie « choisir une alternative ».

Ne pas oublier ce qui est hors des accolades

`inst { inst }` signifie : une instruction obligatoire, puis autant d'instructions supplémentaires que l'on veut. Cela n'autorise pas zéro instruction au total. `BEGIN END` est donc refusé.

`ID { , ID }` signifie : un nom obligatoire, puis éventuellement une virgule et un autre nom, plusieurs fois. Cela accepte `x`, `x,y` et `x,y,z`. Cela refuse une liste vide et la virgule finale `x,`.

`prim { add_op prim }` signifie : un premier élément obligatoire, puis éventuellement un opérateur **avec** un nouvel élément. Cela accepte `x`, `x+2` et `x+2-y`. Cela refuse `x+`, car la répétition complète exige les deux morceaux.

Les parenthèses ont ici deux usages

Dans `prim ::= ( expr )`, les parenthèses sont de vrais terminaux du programme MICRO. Dans une regex ou une description mathématique, des parenthèses peuvent seulement regrouper une notation. Il faut lire la légende et le contexte.

Lien avec les feuilles : pages 32-35. **À retenir :** les règles permettent de déduire les acceptations et les refus avant même de lancer le code.

25. Transformer une règle en fonction C

La méthode du TP2 est appelée **analyse descendante récursive**. « Descendante » : on commence par le programme complet et on descend vers ses constituants. « Récursive » : certaines fonctions peuvent se rappeler indirectement, notamment pour les parenthèses.

Forme dans la grammaire	Traduction dans le programme
Terminal précis	`match(terminal)`
Non-terminal	Appeler sa fonction
A puis B puis C	Trois actions dans cet ordre
Alternative A ou B	Choisir une branche avec `switch` ou `if`
Répétition	Utiliser une boucle

```
program ::= BEGIN inst_list END

static void program(void)
{
    match(BEGIN_TOKEN);
    inst_list();
    match(END_TOKEN);
}
```

Il y a une correspondance directe entre chaque morceau de la recette et chaque instruction C. On vérifie le terminal de début, on reconnaît la construction centrale, puis on vérifie le terminal de fin.

Pourquoi les fonctions sont void ?

Ce parser est un **reconnaisseur**. Si une fonction termine normalement, le morceau attendu a été reconnu. Si la reconnaissance échoue, `syntax_error()` termine le programme. Il n'est donc pas nécessaire, dans ce choix d'implémentation, de renvoyer un arbre ou un résultat de calcul.

La règle de travail de chaque fonction

Quand elle commence, le token courant est le premier token du morceau à reconnaître. Quand elle termine, le token courant est le premier token **après** ce morceau. Elle doit laisser ce token à son appelant.

Par exemple, `expression()` reconnaît `x+2` dans `x+2;` mais laisse `;` courant. C'est la fonction de l'instruction qui doit vérifier et consommer ce point-virgule.

Lien avec le cours : pages 20-21 ; page 34 pour les générateurs comme Bison/Yacc. Ici, nous construisons manuellement les fonctions indiquées par les feuilles.

26. Le token courant : regarder sans consommer

Une grande partie des erreurs de compréhension vient de `next_token()`. Son nom peut donner l'impression qu'il avance. **Dans votre code, il ne fait que consulter le token déjà mémorisé.**

```
static token current_token;

static token next_token(void)
{
    return current_token;
}
```

Une image concrète

Imaginez une carte posée sur une table. Trois personnes peuvent lire son étiquette sans enlever la carte. `next_token()` lit l'étiquette. `match()` vérifie que c'est la bonne carte, puis passe à la suivante.

```
Entrée de tokens : READ LPAREN ID RPAREN SEMICOLON END
Carte actuelle   : READ

inst_list consulte -> READ, toujours actuelle
inst consulte      -> READ, toujours actuelle
match(READ_TOKEN)  -> READ validé, LPAREN devient actuelle
```

Si chaque consultation appelait directement le scanner, `inst_list()` prendrait `READ`, `inst()` prendrait `LPAREN`, et le parser aurait déjà perdu le début de l'instruction. Le contrôle des règles ne fonctionnerait plus.

Qui avance réellement ?

Au début, `main()` appelle une fois `scanner()` pour installer le premier token. Ensuite, `match()` appelle le scanner quand il a consommé avec succès un token ordinaire. Les fonctions comme `inst()` et `expression()` regardent le token courant pour choisir les bonnes branches.

Un seul token d'anticipation

Le parser n'a pas besoin de regarder toute l'instruction. Son premier token distingue les trois branches : `ID`, `READ_TOKEN` et `WRITE_TOKEN`. Il utilise donc une anticipation d'un token pour ces choix.

Lien avec le cours : pages 38 et 100, circulation des tokens entre analyseurs. La mémorisation du token courant est le mécanisme concret qui permet aux fonctions des feuilles d'observer l'entrée sans la perdre.

27. TP2 : son scanner lit les tokens, pas le texte MICRO

Les feuilles supposent qu'une fonction fournit les tokens. Votre version sépare les étapes par un fichier : le TP1 écrit les unités, et le `scanner()` du TP2 lit **les lignes de ce fichier intermédiaire**.

```
Programme MICRO
    -> TP1 : caractères vers tokens
    -> fichier : LIGNE / UNITE / LEXEME
    -> TP2 : lecture des tokens puis contrôle grammatical
```

Le même nom `scanner()` apparaît dans les deux fichiers C, mais les fonctions n'ont pas le même travail interne. Dans le TP1, on reconnaît des lexèmes à partir des caractères. Dans le TP2, on recharge des catégories qui ont déjà été reconnues.

Lire une ligne du fichier intermédiaire

```
while (fgets(line, sizeof line, source_file) != NULL) {
    /* Ignorer les deux lignes d'en-tête. */
    if (sscanf(line, "%u %31s %255[^\n]", &line_number,
               token_name, token_buffer) != 3) {
        /* Ligne invalide : diagnostic et arrêt. */
    }
    value = token_from_name(token_name);
    token_line = line_number;
    last_token_line = line_number;
    return value;
}
```

`fgets()` lit une ligne dans un tableau. `sscanf()` extrait les champs : un numéro avec `%u`, un nom d'unité avec `%31s`, puis le lexème jusqu'au retour à la ligne. Les limites 31 et 255 protègent les tableaux correspondants. Les `&` donnent les adresses où ranger les valeurs numériques.

`token_from_name()` transforme par exemple le texte `REALLITERAL` en la valeur d'énumération `REALLITERAL`. Un nom inconnu provoque une erreur, pas une acceptation silencieuse.

À la fin du fichier, le scanner renvoie `SCANEOF` et vide le lexème. La ligne de fin est estimée par la dernière ligne de token connue : les lignes vides ou commentaires finaux du texte source ne sont plus présents dans le fichier intermédiaire.

Attention : le TP2 attend ce format à trois colonnes. Lui passer directement `program.txt` provoque une erreur de lecture du format, pas une analyse normale du programme MICRO.

28. match() : vérifier, puis avancer exactement une fois

```
static void match(token expected)
{
    if (next_token() != expected) {
        /* Préparer le message d'attente. */
        syntax_error(message);
    }
    if (expected != SCANEOF)
        current_token = scanner();
}
```

Cet extrait simplifié met en évidence les deux actions : **comparer** le token courant au token attendu, puis **avancer** si la comparaison réussit. La version complète, en annexe, construit le message avec `snprintf()` avant d'appeler `syntax_error()`.

Cas réussi

Le courant est `SEMICOLON`, et la fonction appelle `match(SEMICOLON)`. Les catégories sont égales. Le point-virgule est consommé, puis le scanner installe le token suivant.

Cas échoué

Le courant est `END_TOKEN`, alors que l'on attend `SEMICOLON`. Le parser signale « ; attendu ; END rencontré ». Il n'avance pas pour essayer de masquer le problème. La ligne et le lexème incorrects sont encore disponibles au moment de créer le diagnostic.

Cas de fin

`match(SCANEOF)` vérifie que l'entrée est entièrement terminée. Il n'est pas nécessaire de lire encore un token après la fin. Votre code ne rappelle donc pas le scanner pour ce cas.

Pourquoi cela suffit aux fonctions grammaticales ?

Chaque fonction compose ces contrôles. Pour `READ(x);`, elle exige successivement `READ_TOKEN`, `LPAREN`, une liste d'identificateurs, `RPAREN` et `SEMICOLON`. Chaque contrôle réussi déplace la carte courante d'une place.

Le scanner du TP2 ne fait pas lui-même ces contrôles d'ordre. Il pourrait fournir `PLUSOP`, puis `END_TOKEN`, puis `ID`. C'est `match()` et les fonctions de grammaire qui décideront si cette succession est autorisée à l'endroit où elle apparaît.

Lien avec le cours : page 20, contrôle de la forme attendue ; page 100, demande de l'unité suivante. **À retenir :** `next_token()` observe ; `match()` vérifie et consomme ; `scanner()` fournit une nouvelle unité.

29. system_goal() et program() : vérifier tout le fichier

```
static void system_goal(void)
{
    program();
    match(SCANEOF);
}

static void program(void)
{
    match(BEGIN_TOKEN);
    inst_list();
    match(END_TOKEN);
}
```

`program()` applique la structure `BEGIN inst_list END`. `system_goal()` ajoute une contrainte importante : **rien d'utile ne doit rester après le programme**.

Pourquoi END n'est pas la fin de fichier ?

`END` est un mot écrit dans le programme source. `SCANEOF` signifie qu'il n'existe plus de token à lire. On peut tout à fait écrire par erreur `END WRITE(2);` : le mot de fin apparaît, mais l'entrée n'est pas terminée.

```
BEGIN WRITE(1); END
-> program termine ; courant = SCANEOF ; succès

BEGIN WRITE(1); END WRITE(2);
-> program termine ; courant = WRITE ; échec dans system_goal
```

Sans `match(SCANEOF)`, un parser pourrait reconnaître seulement le début correct du fichier et oublier la suite incorrecte. Ce contrôle garantit la reconnaissance du fichier complet.

Et les espaces ou commentaires après END ?

Ils sont permis : le TP1 les élimine et n'écrit aucune ligne de token pour eux. Le TP2 voit donc bien la fin du fichier intermédiaire après `END`.

L'initialisation indispensable

Avant l'appel à `system_goal()`, `main()` exécute `current_token = scanner();`. Ainsi, `program()` peut immédiatement comparer le premier token à `BEGIN_TOKEN`. Sans cette première lecture, le parser n'aurait pas encore installé le premier élément de l'entrée.

Lien avec la feuille : page 32. L'annotation manuscrite près du démarrage suggère aussi la question du token courant ; sa lecture complète n'est pas assez certaine pour en déduire toute une implantation. Le fichier C actuel donne une initialisation explicite et vérifiable.

30. inst_list() : reconnaître une ou plusieurs instructions

```
static void inst_list(void)
{
    inst();
    while (next_token() == ID || next_token() == READ_TOKEN ||
           next_token() == WRITE_TOKEN)
        inst();
}
```

La règle est `inst_list ::= inst { inst }`. Le premier `inst()` est obligatoire. La boucle sert seulement aux instructions supplémentaires. Cela explique le refus de `BEGIN END` : dès le premier appel à `inst()`, `END` n'est pas un début d'instruction.

Pourquoi ces trois catégories ?

Une affectation commence par un identificateur ; une lecture commence par `READ` ; une écriture commence par `WRITE`. Ces trois tokens forment l'ensemble des débuts possibles des instructions.

On peut appeler cet ensemble **FIRST(inst)** : les catégories qui peuvent être au début d'une instruction. Ce nom théorique est un complément utile, mais il n'est pas nécessaire pour comprendre le code : la condition traduit simplement « tant qu'une autre instruction peut commencer ici ».

Exemple avec deux instructions

Après `BEGIN`, le courant est `READ`. Le premier appel reconnaît `READ(x);`. Le courant devient alors `ID` pour `x := 2;`. La boucle voit ce début possible et appelle de nouveau `inst()`. Après cette affectation, le courant est `END`. La boucle s'arrête sans le consommer.

`program()` reprend ensuite et exécute `match(END_TOKEN)`. Chaque fonction reconnaît sa construction et laisse le reste à son appelant.

Que se passe-t-il avec un token inattendu ?

Après une instruction, si le courant est un point-virgule supplémentaire, la boucle s'arrête. `program()` attend alors `END`, mais trouve `;` et signale une erreur. La boucle n'autorise pas ce point-virgule : elle laisse la vérification finale au niveau supérieur.

Lien avec la feuille : page 33. Le `while(true)` et le `switch` de la photo réalisent cette même idée ; votre version utilise une condition de boucle plus directe.

31. inst() : reconnaître une affectation

La fonction `inst()` observe le token courant et choisit une des trois recettes d'instruction. Si elle voit `ID`, elle choisit une affectation.

```
case ID:
    match(ID);
    match(ASSIGNOP);
    expression();
    match(SEMICOLON);
    break;
```

Suivre x := 2 + 3;

1. `match(ID)` **:** vérifier le nom `x`, puis avancer. Le courant devient `ASSIGNOP` pour `:=`.

2. `match(ASSIGNOP)` **:** vérifier l'opérateur et avancer. Le courant devient `INTLITERAL` pour `2`.

3. `expression()` **:** reconnaître l'expression `2 + 3`. Cette fonction consomme les nombres et l'opérateur, puis s'arrête sur `;`.

4. `match(SEMICOLON)` **:** vérifier la fin de l'instruction et avancer vers le début de l'instruction suivante ou vers `END`.

5. `break` **:** sortir du `switch`. On ne doit pas continuer dans la branche `READ` ou `WRITE`.

Pourquoi le nom doit-il être à gauche ?

La grammaire demande `ID := expr ;`. Elle autorise donc `x := 2;`, mais pas `2 := x;`. Un nombre à gauche n'est pas un début d'instruction prévu. Ce refus est déjà syntaxique dans notre grammaire.

Ce que le parser ne fait pas ici

Il ne stocke pas la valeur 5 dans `x`. Il ne cherche pas non plus `x` dans une table des symboles. Il vérifie seulement qu'un identificateur, un opérateur d'affectation, une expression et un point-virgule apparaissent dans cet ordre.

Lien avec le cours : pages 20-21 pour la structure d'une affectation ; pages 22 et 29-30 pour les contrôles de sens et les informations de variables qui dépassent ces TP. **Lien avec la feuille :** page 33.

32. READ et WRITE : deux recettes qui ne demandent pas la même liste

La branche READ

```
case READ_TOKEN:
    match(READ_TOKEN);
    match(LPAREN);
    id_list();
    match(RPAREN);
    match(SEMICOLON);
    break;
```

`READ` prend une liste de noms, comme `READ(x,y);`. On comprend l'idée de lecture : une valeur serait rangée dans chaque variable nommée. La grammaire exige donc des identificateurs, pas des expressions arbitraires.

La branche WRITE

```
case WRITE_TOKEN:
    match(WRITE_TOKEN);
    match(LPAREN);
    expr_list();
    match(RPAREN);
    match(SEMICOLON);
    break;
```

`WRITE` prend une liste d'expressions, comme `WRITE(x, x+2, 3.5);`. Une expression pourrait fournir une valeur à afficher, même si elle n'est pas simplement un nom de variable.

Écriture	Résultat	Raison
`READ(x,y);`	Acceptée	Deux identificateurs
`READ(2);`	Refusée	Le premier élément doit être ID
`READ(x+1);`	Refusée	La liste de noms ne contient pas d'addition
`WRITE(x+1,2.5);`	Acceptée	Deux expressions
`READ();` ou `WRITE();`	Refusée	Au moins un élément obligatoire

Les parenthèses extérieures appartiennent à `inst()`. Les fonctions de liste ne les consomment pas : elles s'arrêtent avant la parenthèse fermante. Cette répartition évite que deux fonctions essaient de consommer le même token.

Si le premier token n'est ni `ID`, ni `READ_TOKEN`, ni `WRITE_TOKEN`, la branche `default` de `inst()` signale qu'une instruction était attendue.

Lien avec la feuille : page 33. **Lien cours-TP :** une structure grammaticale exige des constituants particuliers, pas seulement des caractères individuellement autorisés.

33. id_list() et expr_list() : le mécanisme des virgules

```
static void id_list(void)
{
    match(ID);
    while (next_token() == COMMA) {
        match(COMMA);
        match(ID);
    }
}
```

Le premier identificateur est obligatoire. Ensuite, chaque virgule impose un nouvel identificateur. La virgule ne signifie pas seulement « on peut continuer » ; elle signifie « un autre élément doit maintenant suivre ».

Trace de READ(x,y);

`inst()` consomme `READ` et `(`. `id_list()` consomme `x`. Le courant est la virgule : la boucle consomme `,`, puis exige et consomme `y`. Le courant est maintenant `)` : la boucle termine. `inst()` reprend pour vérifier `)` et `;`.

Pourquoi READ(x,) est refusé

Après `x`, la boucle voit et consomme la virgule. Elle appelle alors `match(ID)`, mais le courant est `RPAREN`. Le diagnostic indique qu'un identificateur était attendu et que `)` a été rencontré.

```
static void expr_list(void)
{
    expression();
    while (next_token() == COMMA) {
        match(COMMA);
        expression();
    }
}
```

La structure est la même. La différence est la nature d'un élément : un simple `ID` pour `id_list()`, une expression entière pour `expr_list()`.

Dans `WRITE(x+2,3.5);`, le premier appel à `expression()` s'arrête sur la virgule. La boucle consomme cette virgule, puis le deuxième appel reconnaît `3.5` et s'arrête sur la parenthèse fermante.

Ce que les deux fonctions laissent à l'appelant

Elles ne consomment ni la parenthèse fermante ni le point-virgule. Quand elles ont reconnu une liste, ces tokens doivent encore être vérifiés par `inst()`. Elles ne reconnaissent pas non plus une liste vide, puisque le premier élément est appelé avant la boucle.

Lien avec la feuille : page 34. **À retenir :** le code « premier élément, puis répétition de virgule + élément » est la traduction directe d'une liste non vide.

34. expression() et add_op() : un opérateur exige un élément après lui

La règle est `expr ::= prim { add_op prim }`. Une expression commence par un primaire, puis peut continuer par des paires « opérateur + primaire ».

```
prim("identificateur, nombre ou expression entre parentheses attendu");
while (next_token() == PLUSOP || next_token() == MINUSOP) {
    token op = next_token();
    /* Construire : operande attendu apres l'operateur. */
    add_op();
    prim(message);
}
```

Le premier `prim()` garantit un élément initial. La boucle s'exécute tant que l'on voit `+` ou `-`. Elle consomme l'opérateur avec `add_op()`, puis demande immédiatement un autre primaire.

Exemple correct : x + 2 - y

Le premier primaire est `x`. La boucle voit `+`, puis reconnaît `2`. Elle voit ensuite `-`, puis reconnaît `y`. Quand elle rencontre `;`, `,` ou `)`, il n'y a plus d'opérateur additif : elle termine en laissant ce token courant.

Exemple incorrect : x + ;

Le premier primaire `x` est valide. `+` est un opérateur valide. Mais après avoir consommé `+`, `prim()` rencontre `;` et ne trouve aucune alternative autorisée. Votre version prépare le message « opérande attendu après '+' » pour expliquer précisément le problème.

```
static void add_op(void)
{
    token tok = next_token();
    if (tok == PLUSOP || tok == MINUSOP)
        match(tok);
    else
        syntax_error("operateur '+' ou '-' attendu");
}
```

Sur la feuille, `add_opp()` déclare une variable `t`, puis utilise `tok` : c'est une incohérence de nom qui doit être corrigée pour compiler. Votre fichier utilise `tok` partout et appelle la fonction `add_op()`.

À retenir : le parser reconnaît la structure, mais n'effectue aucun calcul. Sa boucle ne construit pas non plus un arbre de calcul en mémoire.

35. prim() : les éléments simples et les parenthèses

Un **primaire** est une brique de base d'expression dans cette grammaire. Ce peut être un nom, un entier, un réel ou une expression entre parenthèses.

```
switch (next_token()) {
    case LPAREN:
        match(LPAREN);
        expression();
        match(RPAREN);
        break;
    case ID:          match(ID);          break;
    case INTLITERAL:  match(INTLITERAL);  break;
    case REALLITERAL: match(REALLITERAL); break;
    default:          syntax_error(expected); break;
}
```

Les trois cas simples consomment un seul token. Le cas parenthésé consomme `(`, appelle une nouvelle reconnaissance d'expression, puis exige `)`.

Pourquoi le réel doit être ajouté ici ?

La feuille du TP2, page 35, montre les alternatives `LPAREN`, `ID` et `INTLITERAL`. Après l'extension du TP1, le scanner peut fournir `REALLITERAL`. Sans nouvelle branche, `10.32` serait reconnu au TP1 mais refusé comme primaire au TP2. Votre code contient bien cette branche supplémentaire.

Pourquoi une expression parenthésée est-elle un primaire ?

Cela permet d'utiliser un groupe comme une seule brique à l'extérieur. Dans `(x+2)-y`, le groupe `(x+2)` est le premier primaire de l'expression extérieure, puis vient `-`, puis le primaire `y`.

Trois refus faciles à expliquer

`()` est refusé : après l'ouverture, `expression()` exige un premier primaire, mais rencontre immédiatement la fermeture. `(x+2` est refusé : la fermeture demandée manque. `-2.5` est refusé : `MINUSOP` n'est pas une alternative initiale de `prim()`.

Le paramètre `expected` est une chaîne destinée au message d'erreur. Il ne change pas les formes acceptées. Il permet au même contrôle de dire soit « expression attendue », soit « opérande attendu après - » selon l'endroit qui a appelé `prim()`.

Lien cours-TP : pages 20-21, structure hiérarchique ; feuille page 35 ; extension du TP1 pour `REALLITERAL`.

36. La récursion expliquée avec des parenthèses

La récursion apparaît parce que `expression()` appelle `prim()`, et que le cas parenthésé de `prim()` appelle à son tour `expression()`. Ce n'est pas une répétition infinie : le cas parenthésé consomme d'abord une ouverture, puis les appels simples consomment les éléments internes.

```
Texte : (x + 2) - y

expression extérieure
  prim extérieure : rencontre (
    consommer (
    expression intérieure
      prim : consommer x
      add_op : consommer +
      prim : consommer 2
      courant = ) ; terminer l'expression intérieure
    consommer )
  add_op extérieure : consommer -
  prim extérieure : consommer y
```

Comment la fonction sait-elle où revenir ?

Quand une fonction appelle une autre fonction, C conserve l'endroit où reprendre sur la **pile d'appels**. La pile fonctionne comme une pile d'assiettes : le dernier appel commencé est le premier qui doit terminer.

Avec `((x+2)-y)`, on conserve plusieurs niveaux ouverts. La fermeture la plus intérieure permet de terminer l'appel intérieur. On revient ensuite au niveau précédent pour traiter sa suite et sa fermeture.

Pourquoi expression() ne consomme-t-elle pas la fermeture ?

Elle s'arrête dès qu'elle ne voit plus `+` ou `-` après un primaire. La parenthèse fermante reste courante. Le `prim()` qui a consommé l'ouverture possède la responsabilité de consommer la fermeture correspondante.

Le cours cite les automates à pile pour l'analyse syntaxique à la page 20. Dans ce programme, nous utilisons la pile d'appels du langage C pour gérer l'imbrication des reconnaissances ; nous ne programmons pas un automate à pile explicite.

Limite pratique : des parenthèses extrêmement profondes peuvent consommer beaucoup de pile. Le programme actuel ne définit pas de limite de profondeur personnalisée. Cela ne gêne pas les exemples ordinaires des TP.

37. Un programme complet : du texte aux 20 tokens

Reprenons le fichier actuel `TP1/program.txt`. Il est utile de vérifier d'abord le TP1, puis de présenter sa sortie au TP2.

```
BEGIN
READ(nombre);
resultat := nombre + 10.32 - 2;
WRITE(resultat);
END
```

N°	Ligne	Catégorie	Lexème
1	1	BEGIN	BEGIN
2	2	READ	READ
3	2	LPAREN	(
4	2	ID	nombre
5	2	RPAREN	)
6	2	SEMICOLON	;
7	3	ID	resultat
8	3	ASSIGNOP	:=
9	3	ID	nombre
10	3	PLUSOP	+
11	3	REALLITERAL	10.32
12	3	MINUSOP	-
13	3	INTLITERAL	2
14	3	SEMICOLON	;
15	4	WRITE	WRITE
16	4	LPAREN	(
17	4	ID	resultat
18	4	RPAREN	)
19	4	SEMICOLON	;
20	5	END	END

`SCANEOF` arrive ensuite à l'interface du scanner, mais n'est pas une ligne écrite par le TP1. Les espaces et retours à la ligne ont disparu comme unités, tout en servant au suivi des numéros de ligne.

Le mot `nombre` apparaît deux fois comme lexème `ID`. Le réel `10.32` est une seule unité. L'opérateur `:=` est également une seule unité, bien qu'il contienne deux caractères.

Vérification réelle : après génération d'un fichier neuf à partir de cette source, les deux programmes terminent avec succès. La suite montrée ici contient exactement un point-virgule par instruction.

38. Trace du TP2 : vérifier ce programme complet

`main()` installe le token 1, `BEGIN`, puis appelle `system_goal()`. La trace ci-dessous montre les grandes étapes. Chaque `match()` ordinaire réussi avance au token suivant.

Étape	Tokens reconnus	Token laissé courant
`program` vérifie le début	1 : BEGIN	2 : READ
Première `inst` : branche READ	2 READ, 3 (	4 ID
`id_list`	4 : nombre	5 :)
Fin de READ	5), 6 ;	7 : ID resultat
Deuxième `inst` : affectation	7 ID, 8 :=	9 : ID nombre
Premier `prim` de l'expression	9 : nombre	10 : +
Première paire opérateur-primaire	10 +, 11 réel 10.32	12 : -
Deuxième paire opérateur-primaire	12 -, 13 entier 2	14 : ;
Fin de l'affectation	14 : ;	15 : WRITE
Troisième `inst` : branche WRITE	15 WRITE, 16 (	17 : ID resultat
`expr_list` et son expression	17 : resultat	18 :)
Fin de WRITE	18), 19 ;	20 : END
`inst_list` termine	Aucun token consommé	20 : END
`program` vérifie la fin	20 : END	SCANEOF
`system_goal` vérifie tout le fichier	SCANEOF	Fin confirmée

Ce qu'il faut observer

`id_list()` laisse `)` au niveau instruction. `expression()` laisse `;` au niveau affectation. `inst_list()` laisse `END` au niveau programme. Chaque fonction s'arrête à la frontière de son propre morceau.

La consultation du token courant peut se produire plusieurs fois dans une ligne de code ou entre plusieurs fonctions. Cela ne consomme rien. Seul un `match()` réussi provoque ici la lecture d'un nouveau token ordinaire.

Ce que le résultat nous apprend

L'exemple respecte le début, les trois instructions et la fin. Il utilise des entiers et un réel selon les modèles reconnus. Le succès du TP2 valide cette structure. Il ne donne pas la valeur finale de `resultat`, car `nombre` n'est pas lu et l'expression n'est pas évaluée par ce programme.

Lien avec le cours : pages 20-21 et 100. **Lien avec les TP :** le TP1 prépare exactement les éléments que chaque appel du TP2 attend.

39. Votre exemple du TP2 : comprendre la première erreur

Le fichier actuel `TP2/program.txt` contient :

```
BEGIN
READ(x, y);
resultat := (x + 10.32) - ;
WRITE(resultat, x + );
END
```

Le TP1 reconnaît tous les caractères et produit les tokens. Il réussit, car aucune unité n'est illégale. Le TP2 échoue ensuite à la ligne 3.

Le chemin exact vers l'erreur

L'affectation commence correctement par `ID` puis `ASSIGNOP`. L'expression rencontre `(`, reconnaît `x + 10.32`, puis ferme `)`. Ce groupe forme un primaire complet. Ensuite, la boucle de l'expression extérieure voit `MINUSOP`.

Elle prépare le message « opérande attendu après '-' », consomme `-`, puis appelle `prim(message)`. Le courant est `SEMICOLON`. Or un primaire doit commencer par un nom, un nombre ou `(`. Le point-virgule ne correspond à aucun de ces choix.

```
Ligne 3 : operande attendu apres '-' ; ';' rencontre
```

Pourquoi l'erreur de la ligne 4 n'apparaît-elle pas immédiatement ?

`syntax_error()` appelle `exit(EXIT_FAILURE)`. L'analyse s'arrête à la première erreur. La ligne 4 contient aussi un opérande manquant après `+`, mais le parser ne l'atteint pas dans cette exécution.

Voici une **correction possible**, choisie pour l'exemple. On ne peut pas connaître les valeurs que vous souhaitiez mettre à la place des éléments absents.

```
BEGIN
READ(x, y);
resultat := (x + 10.32) - y;
WRITE(resultat, x + 1);
END
```

Cette proposition respecte la grammaire. Elle illustre la correction ; elle ne remplace pas vos fichiers existants. Les éléments `y` et `1` ont été ajoutés pour rendre chaque opérateur suivi d'un primaire.

40. Comprendre un diagnostic au lieu de deviner la correction

Un diagnostic syntaxique contient trois informations utiles : le numéro de ligne où le problème est découvert, la catégorie attendue et le lexème rencontré.

```
';' attendu ; 'END' rencontre
```

Cela ne signifie pas qu'il faut supprimer `END`. Cela signifie qu'au moment de terminer une instruction, le parser n'a pas trouvé son point-virgule. Le caractère à ajouter peut se trouver sur la ligne précédente.

Diagnostic ou situation	Cause probable à vérifier
Opérande attendu après +, rencontre `;`	Expression interrompue après l'opérateur
Identificateur attendu, rencontre `)`	Liste READ vide ou virgule finale
`)` attendu, rencontre `;`	Parenthèse fermante manquante
Fin de fichier attendue, rencontre WRITE	Texte utile après END
END attendu, rencontre `;`	Point-virgule supplémentaire après une instruction

Un cas réel dans votre dossier

Le fichier déjà présent `TP1/unites_lexicales.txt` contient **trois points-virgules supplémentaires** à la ligne 4 après la fin de `WRITE(resultat);`. Le `TP1/program.txt` actuel ne contient pas ces trois symboles.

Si l'on passe ce fichier de tokens existant au TP2, il refuse l'entrée : après la troisième instruction, il attend `END` et rencontre un autre `;`. Si l'on régénère un fichier de tokens neuf depuis le programme actuel du TP1, les deux étapes réussissent.

On peut constater cette différence, mais on ne peut pas déduire avec certitude comment le fichier de tokens a été modifié ou à partir de quelle ancienne source il a été créé.

Une méthode simple de recherche

Vérifiez d'abord que vous analysez le bon fichier. Vérifiez ensuite le succès du TP1. Regardez les tokens autour de la ligne signalée. Reconstituez enfin la recette de l'instruction : début, constituants et fin. Corrigez la première erreur, régénérez les tokens, puis relancez le TP2.

Lien avec le cours : pages 31, 38 et 107. Un bon diagnostic indique le point de détection ; il ne devine pas toujours l'intention du programmeur.

41. Exécuter les deux étapes correctement sous PowerShell

Ces commandes sont à lancer depuis le dossier racine **TP**. Elles utilisent GCC, présent dans l'environnement de vérification.

```
gcc -std=c11 -Wall -Wextra -Wpedantic TP1/tp1.c -o TP1/tp1.exe
gcc -std=c11 -Wall -Wextra -Wpedantic TP2/tp2.c -o TP2/tp2.exe

.\TP1\tp1.exe .\TP1\program.txt .\tmp\tokens_exemple.txt
if ($LASTEXITCODE -eq 0) {
    .\TP2\tp2.exe .\tmp\tokens_exemple.txt
}
```

Le dossier `tmp` existe dans la livraison de ce guide. Si vous utilisez ces commandes ailleurs, créez d'abord le dossier choisi pour la sortie. Vous pouvez aussi choisir un autre chemin de fichier existant.

Lire les arguments

Pour le TP1, le premier argument est la source MICRO. Le second est le fichier où écrire les tokens. Si ce second argument est omis, le programme écrit `unites_lexicales.txt` **dans le dossier courant**, pas automatiquement dans le dossier TP1.

Pour le TP2, l'unique argument est le fichier de tokens à trois colonnes. Il faut transmettre le même chemin que celui utilisé pour la sortie du TP1.

`$LASTEXITCODE` contient le code de retour du dernier exécutable. Zéro signifie ici succès. La condition évite de présenter au parser des tokens issus d'une entrée lexicalement incorrecte.

Que signifie compiler avec les avertissements ?

`-std=c11` choisit la norme C11. `-Wall`, `-Wextra` et `-Wpedantic` demandent des avertissements utiles. Pour la vérification de ce guide, `-Werror` a aussi été utilisé : les avertissements éventuels deviennent des échecs de compilation.

Compiler correctement le C ne prouve pas que la grammaire est bien appliquée. Il faut aussi lancer des exemples valides et invalides. Les pages suivantes présentent les résultats réellement observés.

À retenir : compiler l'outil, lancer le TP1, vérifier son succès, puis lancer le TP2 sur la sortie produite. La précision des chemins évite beaucoup de faux problèmes.

42. Les essais réalisés et ce qu'ils vérifient

Les deux fichiers actuels ont été compilés avec `-std=c11 -Wall -Wextra -Wpedantic -Werror`. Vingt-cinq entrées MICRO ont ensuite été examinées, ainsi que le fichier de tokens déjà présent. Les essais ont utilisé des copies de sortie dans un dossier de vérification.

Cas, dans BEGIN ... END si nécessaire	TP1	TP2 si TP1 réussit
Programme actuel du TP1	Succès	Succès
Programme actuel du TP2	Succès	Échec : opérande après -
`WRITE(5.2,.5,5.,0.25);`	Succès	Succès
Parenthèses imbriquées et listes	Succès	Succès
Commentaire après END sans retour final	Succès	Succès
`x := 10 + ;`	Succès	Échec : opérande absent
`x := -2.5;`	Succès	Échec : primaire initial
`x := 1e3;`	Succès	Échec : ID e3 après entier
`x := 5.2.3;`	Succès	Échec : second réel .3
`x := 12abc;`	Succès	Échec : ID abc après entier
`READ();` et `READ(2);`	Succès	Échec : ID attendu
`READ(x,);`	Succès	Échec : ID après virgule
`WRITE();`	Succès	Échec : expression attendue
`BEGIN END`	Succès	Échec : instruction obligatoire
Instruction après END	Succès	Échec : fin de fichier
`x := 5--2;` puis END ligne suivante	Succès	Échec : ; mangé par commentaire
`x := 5 - 2;`	Succès	Succès
Point-virgule ou parenthèse manquants	Succès	Échec syntaxique
END absent en fin de fichier	Succès	Échec : END attendu
`begin` en minuscules	Succès	Échec : BEGIN attendu
`WRITE(.);` et `WRITE(1@);`	Échec	Non lancé
Identificateur de 256 caractères	Échec	Non lancé

Ces essais vérifient des comportements représentatifs. Ils ne prouvent pas tous les cas possibles. Le fichier de tokens déjà présent est aussi refusé à cause des trois ; supplémentaires ; un fichier fraîchement généré depuis la source du TP1 est accepté.

43. Relier chaque notion du cours à ce que nous avons fait

Notion du cours	Dans le TP1	Dans le TP2
Caractères vers unités, p. 18 et 38	`scanner()` lit avec `read_char()`	Reçoit les unités déjà produites
Lexème et modèle, p. 39-40	`token_buffer`, règles ID et nombres	Garde le lexème pour les diagnostics
Expressions régulières, p. 64-78	Boucles qui réalisent ID et nombres	Les règles grammaticales sont différentes
Automates finis, p. 79-85 et 100	Branches et drapeau `is_real`	Ne suffisent pas seuls pour l'imbrication générale
Structure grammaticale, p. 20-21	Ne vérifie pas l'ordre des unités	Fonctions `program`, `inst`, `expression`
Imbrication, p. 20-21	Reconnaît `(` et `)` individuellement	Récursion entre `prim` et `expression`
Gestion des lignes, p. 38	Compte les retours à la ligne	Relit la ligne d'origine des tokens
Gestion des erreurs, p. 31 et 107	Signale, compte et peut poursuivre	S'arrête au premier échec syntaxique
Table des symboles, p. 29-30	Non construite ici	Non utilisée ici
Sémantique, p. 22	Non réalisée	Non réalisée
Traduction et génération, p. 23-27	Non réalisées	Non réalisées

Pourquoi deux analyses au lieu d'une seule ?

Le scanner résout les questions locales : combien de caractères forment le nom ? Est-ce `-` ou `--` ? L'analyseur syntaxique résout les questions de structure : est-ce une instruction complète ? La fermeture correspond-elle à une expression ouverte ?

Le fichier intermédiaire rend cette séparation visible. On peut regarder les unités avant de contrôler la grammaire. Dans un compilateur réel, on peut aussi appeler directement le scanner depuis le parser : le rôle des étapes reste le même, même sans fichier intermédiaire.

Où se situe Flex ?

Aux pages 93-99, le cours montre qu'un outil peut générer le scanner à partir de regex. Ici, vous avez écrit le scanner à la main. Flex et notre code peuvent réaliser le même type de mission ; les TP fournis n'exigent pas de remplacer le code par un fichier Flex.

À retenir : le cours donne les idées et les règles ; le C donne les tableaux, les boucles, les branches et les appels qui rendent ces idées exécutables.

44. Les différences, corrections et limites à connaître

Les points importants dans les feuilles

- `eof(stdin)` n'est pas du C standard ; votre version teste le résultat de la lecture puis renvoie `SCANEOF`.
- Le commentaire doit aussi s'arrêter sur EOF ; votre version le fait.
- `add_opp()` présente `t` puis `tok` sur la photo ; votre code emploie `tok` de façon cohérente.
- Le type `token`, les fonctions de tampon, `check_reserved`, `match` et le mécanisme d'anticipation ne sont pas tous définis dans les photos. Votre code les rend explicites.
- L'extension des réels demande `REALLITERAL` au TP1 **et** une branche correspondante de `prim()` au TP2.

Les limites de la version actuelle

Les signes unaires, les exposants, la multiplication et la division ne sont pas acceptés. Les identificateurs ne sont pas pris en charge comme texte Unicode complet ; le code s'appuie sur les fonctions C de classification et ne règle pas explicitement la locale.

Le TP1 peut ignorer un caractère illégal et poursuivre. Il faut respecter son code de retour avant le TP2. Le TP2 ne reverifie pas que chaque lexème correspond bien au modèle annoncé par sa catégorie : il fait confiance aux lignes produites correctement par le TP1.

`token_from_name()` associe les noms à leurs valeurs grâce à l'ordre de son tableau et à l'ordre de l'énumération. Si l'on ajoute ou déplace une catégorie, les deux ordres doivent rester compatibles.

Le TP2 s'arrête à la première erreur. En afficher plusieurs demanderait une récupération contrôlée, par exemple en cherchant un point de reprise comme `;`. Continuer simplement après chaque erreur pourrait produire des diagnostics trompeurs.

Pourquoi ne pas tout ajouter tout de suite ?

Ces limites correspondent au périmètre pédagogique. Ajouter `*` et `/` demanderait aussi une grammaire de priorités. Ajouter `-2.5` demanderait une règle pour le signe unaire. Ajouter `1e3` demanderait d'étendre la reconnaissance numérique.

Comprendre précisément la version actuelle est la première étape. Une modification doit ensuite préserver la cohérence **modèle -> scanner -> tokens -> grammaire -> parser**.

45. Exercices : vérifier que les idées sont comprises

Répondez d'abord sans lire la correction de la page suivante. Pour chaque refus, précisez si le problème apparaît au TP1 ou au TP2.

1. Découper sans exécuter

Quels sont les tokens et lexèmes de `total_2 := .5 + 12;` ? Combien de tokens ordinaires obtient-on ? Les espaces sont-ils des tokens ?

2. Distinguer catégorie et texte

`x` et `resultat` sont-ils le même lexème ? Ont-ils la même catégorie ? Pourquoi cela simplifie-t-il la grammaire des affectations ?

3. Comprendre une liste

Pourquoi `READ(x,y);` est-il accepté, alors que `READ(x,);` est refusé ? À quel appel la seconde écriture échoue-t-elle ?

4. Identifier le niveau d'erreur

Comparez `x := 2 @ 3;`, `x := 2 + ;` et `x := y + 1;`. Que peuvent dire nos deux analyseurs ? Que ne peuvent-ils pas dire ?

5. Suivre le token courant

Le courant est `READ_TOKEN`. On appelle trois fois `next_token()`. Quelle est sa valeur ensuite ? Puis on appelle `match(READ_TOKEN)`. Quel token doit devenir courant pour `READ(x);` ?

6. Comprendre la fin d'une expression

Dans `WRITE(x+2,3);`, quel token reste courant quand le premier appel à `expression()` termine ? Quelle fonction le consomme ?

7. Anticiper les limites

Que produit le TP1 pour `1e3` ? Pour `5.2.3` ? Pourquoi le TP2 refuse-t-il `x := 1e3;` et `x := 5.2.3;` ?

8. Étendre la grammaire avec méthode

On souhaite reconnaître un réel dans une expression alors que `prim()` ne contient encore que ID et INTLITERAL. Quelle modification est indispensable ? Pourquoi l'ajout au scanner seul ne suffit-il pas ?

9. Comprendre ce qui a été fait

Expliquez en cinq phrases : le rôle du TP1, sa sortie, le rôle du TP2, la différence entre `next_token()` et `match()`, et le sens limité du message de succès.

46. Corrections des exercices, expliquées

1. On obtient six tokens ordinaires : `ID("total_2")`, `ASSIGNOP(":=")`, `REALLITERAL(".5")`, `PLUSOP("+")`, `INTLITERAL("12")`, `SEMICOLON(";")`. Les espaces sont ignorés. `SCANEOF` arrive ensuite si le fichier se termine, mais n'est pas un lexème ordinaire.

2. Les lexèmes sont différents : les textes exacts sont `x` et `resultat`. La catégorie est la même : ID. Une seule règle `ID := expr ;` convient donc à tous les noms conformes au modèle.

3. `id_list()` reconnaît d'abord `x`. Après une virgule, elle exige encore un ID. Dans `READ(x,)`, `match(ID)` rencontre `RPAREN` et échoue. La virgule ne peut pas terminer la liste.

4. `@` déclenche une erreur lexicale au TP1. Le `+` sans opérande passe le TP1, mais échoue au TP2. `x := y + 1;` respecte la grammaire ; nos outils ne vérifient pas si `y` existe ou possède une valeur.

5. Après trois consultations, le courant reste `READ_TOKEN`. Après `match(READ_TOKEN)` réussi, `LPAREN` devient courant. Regarder plusieurs fois ne déplace pas la carte ; consommer le fait avancer.

6. La virgule reste courante après `x+2`. `expr_list()` la consomme avec `match(COMMA)`, puis appelle `expression()` pour reconnaître `3`. `inst()` consommera ensuite `)` et `;`.

7. `1e3` donne un entier `1`, puis un identificateur `e3`. `5.2.3` donne le réel `5.2`, puis le réel `.3`. Après le premier primaire, il n'y a aucun opérateur additif : l'expression termine. L'affectation attend alors `;` et rencontre la deuxième unité, d'où le refus.

8. Il faut ajouter `case REALLITERAL: match(REALLITERAL); break;` à `prim()`, en conservant les catégories compatibles avec le scanner. Reconnaître un élément au TP1 ne l'autorise pas automatiquement dans une règle du TP2.

9. Le TP1 lit des caractères et les regroupe en unités. Il écrit leur ligne, catégorie et lexème. Le TP2 lit ces unités et vérifie la grammaire MICRO. `next_token()` consulte, tandis que `match()` vérifie et avance. Un succès signifie que la structure est conforme, sans exécuter le programme ni contrôler tous ses types.

47. Fiche de révision : savoir expliquer les deux TP

Les cinq mots essentiels

Caractère : un élément du texte. **Lexème :** le morceau exact reconnu. **Modèle :** la règle de forme d'un lexème. **Token :** la représentation d'une unité reconnue, ici une catégorie et des informations séparées. **Grammaire :** les règles d'organisation des unités.

TP1 : la réponse à donner

Le scanner lit de gauche à droite. Il ignore les blancs et les commentaires. Il reconnaît les mots, les nombres, les opérateurs et les séparateurs. Il conserve le lexème dans un tampon et retourne une catégorie par appel. Le fichier de sortie conserve aussi la ligne.

L'exercice demande d'ajouter les réels. Nous définissons leur forme, ajoutons REALLITERAL et adaptons la branche numérique pour lire un point et ses chiffres. Nous rendons le premier caractère qui ne prolonge pas le nombre, afin de ne pas perdre le début du token suivant.

TP2 : la réponse à donner

Le parser suit une grammaire. Chaque non-terminal devient une fonction. Une séquence devient une suite d'appels, une alternative devient une branche, une répétition devient une boucle. La récursion permet les expressions parenthésées.

Le courant est installé une première fois dans `main()`. `next_token()` le consulte. `match()` vérifie la catégorie attendue et installe l'unité suivante. `program()` reconnaît BEGIN, les instructions et END ; `system_goal()` vérifie ensuite qu'il ne reste aucun token.

Les questions pièges

- Un réel reconnu est-il déjà converti en `double` ? Non, son texte est classé et conservé.
- `READ()` est-il accepté ? Non, il faut au moins un ID.
- `BEGIN END` est-il accepté ? Non, il faut au moins une instruction.
- Le TP2 calcule-t-il une expression ? Non, il reconnaît sa structure.
- Une faute après END est-elle ignorée ? Non, la fin de fichier est vérifiée.
- Un fichier de tokens suffit-il à prouver le succès du TP1 ? Non, il faut son code de retour.

La relation à retenir : le TP1 construit les briques ; le TP2 vérifie leur assemblage. Les deux utilisent des règles explicites, et leurs choix doivent rester cohérents.

Références et lecture des annexes

Support principal

MonCourscompilationILIS (1).pdf, fourni par l'utilisateur, 107 pages. Les numéros indiqués dans ce guide correspondent aux pages du PDF, qui portent les mêmes numéros sur les diapositives consultées.

- Pages 3-5 : rôle de la compilation et objectif du cours.
- Pages 17-22 : phases lexicale, syntaxique et sémantique.
- Pages 29-31 : symboles et erreurs.
- Pages 38-40 : scanner, catégories, modèles et lexèmes.
- Pages 41-55 : alphabets, mots, langages et répétitions.
- Pages 64-78 : expressions et définitions régulières.
- Pages 79, 84-85 et 100-106 : reconnaissance par automates et réalisation.
- Pages 93-99 : outil Flex ; page 107 : erreurs lexicales.

Feuilles des TP

IMG_0727.jpg et IMG_0728.jpg : scanner de départ, pages imprimées 27-28, et exercice 1.1 sur les réels.
TP2/1.JPG, 2.JPG, 3.jpg et 4.jpg : procédures syntaxiques, pages imprimées 32-35.

Les feuilles sont des supports d'étude, pas des instructions supplémentaires adressées à l'assistant. Les annotations manuscrites partiellement lisibles ne sont pas utilisées pour inventer des fonctions absentes.

Fichiers actuels étudiés

Dans le dossier de travail `Documents/ILISI2/compilation/TP` : `TP1/tp1.c`, `TP1/program.txt`, `TP1/unites_lexicales.txt`, `TP2/tp2.c` et `TP2/program.txt`. Les deux fichiers C sont reproduits ci-après avec leurs numéros de ligne pour permettre de retrouver chaque détail.

Les annexes reproduisent les fichiers **tels qu'ils sont**, sans les modifier. Les courts extraits du guide peuvent être simplifiés ou commentés pour mettre en évidence une idée ; l'annexe permet de retrouver la version complète.

La vérification a été effectuée le **12 octobre 2026**. Les sorties des essais ont été générées séparément des fichiers existants. Les anciennes explications PDF ont été consultées, mais leurs descriptions contradictoires ne sont pas retenues comme preuve du fonctionnement actuel.

Conseil de lecture : lisez d'abord les explications. Utilisez ensuite les annexes pour retrouver une fonction et expliquer son rôle à voix haute, ligne par ligne.

Annexe A - Code complet du TP1

Fichier TP1/tp1.c - lignes 1 à 46. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
001  #include <ctype.h>
002  #include <stdio.h>
003  #include <stdlib.h>
004  #include <string.h>
005
006  #define TOKEN_BUFFER_SIZE 256
007
008  typedef enum {
009      BEGIN_TOKEN,
010      END_TOKEN,
011      READ_TOKEN,
012      WRITE_TOKEN,
013      ID,
014      INTLITERAL,
015      REALLITERAL,
016      LPAREN,
017      RPAREN,
018      SEMICOLON,
019      COMMA,
020      ASSIGNOP,
021      PLUSOP,
022      MINUSOP,
023      SCANEOF
024  } token;
025
026  static FILE *source_file;
027  static const char *source_name;
028  static char token_buffer[TOKEN_BUFFER_SIZE];
029  static size_t token_buffer_index;
030  static unsigned int current_line = 1;
031  static unsigned int token_line = 1;
032  static unsigned int error_count = 0;
033
034  static void clear_buffer(void)
035  {
036      token_buffer_index = 0;
037      token_buffer[0] = '\0';
038  }
039
040  static void buffer_char(int c)
041  {
042      if (token_buffer_index + 1 >= TOKEN_BUFFER_SIZE) {
043          fprintf(stderr, "%s:%u: erreur: unite lexicale trop longue\n",
044                  source_name, token_line);
045          exit(EXIT_FAILURE);
046      }
```

Annexe A - Code complet du TP1 - suite 1

Fichier TP1/tp1.c - lignes 47 à 92. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
047
048     token_buffer[token_buffer_index++] = (char)c;
049     token_buffer[token_buffer_index] = '\0';
050 }
051
052 static int read_char(void)
053 {
054     int c = fgetc(source_file);
055
056     if (c == '\n')
057         ++current_line;
058
059     return c;
060 }
061
062 static void unread_char(int c)
063 {
064     if (c == EOF)
065         return;
066
067     if (c == '\n')
068         --current_line;
069
070     ungetc(c, source_file);
071 }
072
073 static token check_reserved(void)
074 {
075     if (strcmp(token_buffer, "BEGIN") == 0)
076         return BEGIN_TOKEN;
077     if (strcmp(token_buffer, "END") == 0)
078         return END_TOKEN;
079     if (strcmp(token_buffer, "READ") == 0)
080         return READ_TOKEN;
081     if (strcmp(token_buffer, "WRITE") == 0)
082         return WRITE_TOKEN;
083
084     return ID;
085 }
086
087 static void lexical_error(int c)
088 {
089     if (isprint((unsigned char)c))
090         fprintf(stderr, "%s:%u: erreur: caractere inattendu '%c'\n",
091                 source_name, token_line, c);
092     else
```

Annexe A - Code complet du TP1 - suite 2

Fichier TP1/tp1.c - lignes 93 à 138. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
093         fprintf(stderr, "%s:%u: erreur: caractere inattendu 0x%02X\n",
094                 source_name, token_line, (unsigned int)(unsigned char)c);
095
096     ++error_count;
097 }
098
099 static const char *token_name(token value)
100 {
101     switch (value) {
102         case BEGIN_TOKEN: return "BEGIN";
103         case END_TOKEN:   return "END";
104         case READ_TOKEN:  return "READ";
105         case WRITE_TOKEN: return "WRITE";
106         case ID:           return "ID";
107         case INTLITERAL:   return "INTLITERAL";
108         case REALLITERAL:  return "REALLITERAL";
109         case LPAREN:       return "LPAREN";
110         case RPAREN:       return "RPAREN";
111         case SEMICOLON:    return "SEMICOLON";
112         case COMMA:        return "COMMA";
113         case ASSIGNOP:     return "ASSIGNOP";
114         case PLUSOP:       return "PLUSOP";
115         case MINUSOP:      return "MINUSOP";
116         case SCANEOF:      return "SCANEOF";
117     }
118
119     return "UNKNOWN";
120 }
121
122 static token scanner(void)
123 {
124     int in_char;
125     int c;
126
127     while ((in_char = read_char()) != EOF) {
128         if (isspace((unsigned char)in_char))
129             continue;
130
131         clear_buffer();
132         token_line = current_line;
133
134         if (isalpha((unsigned char)in_char)) {
135             buffer_char(in_char);
136
137             for (c = read_char();
138                  c != EOF && (isalnum((unsigned char)c) || c == '_');
```

Annexe A - Code complet du TP1 - suite 3

Fichier TP1/tp1.c - lignes 139 à 184. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
139                  c = read_char())
140                 buffer_char(c);
141
142             unread_char(c);
143             return check_reserved();
144         }
145
146         if (isdigit((unsigned char)in_char) || in_char == '.') {
147             int is_real = in_char == '.';
148
149             buffer_char(in_char);
150
151             if (in_char == '.') {
152                 c = read_char();
153                 if (c == EOF || !isdigit((unsigned char)c)) {
154                     unread_char(c);
155                     lexical_error(in_char);
156                     continue;
157                 }
158                 buffer_char(c);
159             }
160
161             for (c = read_char();
162                  c != EOF && isdigit((unsigned char)c);
163                  c = read_char())
164                 buffer_char(c);
165
166             if (!is_real && c == '.') {
167                 is_real = 1;
168                 buffer_char(c);
169                 for (c = read_char();
170                      c != EOF && isdigit((unsigned char)c);
171                      c = read_char())
172                     buffer_char(c);
173             }
174
175             unread_char(c);
176             return is_real ? REALLITERAL : INTLITERAL;
177         }
178
179         buffer_char(in_char);
180
181         switch (in_char) {
182             case '(':
183                 return LPAREN;
184             case ')':
```

Annexe A - Code complet du TP1 - suite 4

Fichier TP1/tp1.c - lignes 185 à 230. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
185                 return RPAREN;
186             case ';':
187                 return SEMICOLON;
188             case ',':
189                 return COMMA;
190             case '+':
191                 return PLUSOP;
192 
193             case ':':
194                 c = read_char();
195                 if (c == '=') {
196                     buffer_char(c);
197                     return ASSIGNOP;
198                 }
199                 unread_char(c);
200                 lexical_error(in_char);
201                 break;
202 
203             case '-':
204                 c = read_char();
205                 if (c == '-') {
206                     do {
207                         in_char = read_char();
208                     } while (in_char != '\n' && in_char != EOF);
209                 } else {
210                     unread_char(c);
211                     return MINUSOP;
212                 }
213                 break;
214 
215             default:
216                 lexical_error(in_char);
217                 break;
218         }
219     }
220 
221     clear_buffer();
222     token_line = current_line;
223     return SCANEOF;
224 }
225 
226 int main(int argc, char *argv[])
227 {
228     const char *output_name;
229     FILE *output_file;
230     token current_token;
```

Annexe A - Code complet du TP1 - suite 5

Fichier TP1/tp1.c - lignes 231 à 274. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
231
232     if (argc < 2 || argc > 3) {
233         fprintf(stderr, "Utilisation: %s <fichier-source> [fichier-tokens]\n",
234                 argv[0]);
235         return EXIT_FAILURE;
236     }
237
238     source_name = argv[1];
239     output_name = argc == 3 ? argv[2] : "unites_lexicales.txt";
240
241     source_file = fopen(source_name, "r");
242     if (source_file == NULL) {
243         fprintf(stderr, "%s: erreur: impossible d'ouvrir le fichier source\n",
244                 source_name);
245         return EXIT_FAILURE;
246     }
247
248     output_file = fopen(output_name, "w");
249     if (output_file == NULL) {
250         fprintf(stderr, "%s: erreur: impossible de creer le fichier\n",
251                 output_name);
252         fclose(source_file);
253         return EXIT_FAILURE;
254     }
255
256     fprintf(output_file, "%-8s %-16s %s\n", "LIGNE", "UNITE", "LEXEME");
257     fprintf(output_file, "%-8s %-16s %s\n", "-----", "-----", "------");
258
259     while ((current_token = scanner()) != SCANEOF) {
260         fprintf(output_file, "%-8u %-16s %s\n",
261                 token_line, token_name(current_token), token_buffer);
262     }
263
264     fclose(output_file);
265     fclose(source_file);
266
267     if (error_count != 0) {
268         fprintf(stderr, "%u erreur(s) lexicale(s) detectee(s).\n", error_count);
269         return EXIT_FAILURE;
270     }
271
272     printf("Analyse terminee: %s genere.\n", output_name);
273     return EXIT_SUCCESS;
274 }
```

Annexe B - Code complet du TP2

Fichier TP2/tp2.c - lignes 1 à 50. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
001  #include <stdio.h>
002  #include <stdlib.h>
003  #include <string.h>
004
005  #define TOKEN_BUFFER_SIZE 256
006
007  typedef enum {
008      BEGIN_TOKEN,
009      END_TOKEN,
010      READ_TOKEN,
011      WRITE_TOKEN,
012      ID,
013      INTLITERAL,
014      REALLITERAL,
015      LPAREN,
016      RPAREN,
017      SEMICOLON,
018      COMMA,
019      ASSIGNOP,
020      PLUSOP,
021      MINUSOP,
022      SCANEOF
023  } token;
024
025  static FILE *source_file;
026  static const char *source_name;
027  static char token_buffer[TOKEN_BUFFER_SIZE];
028  static unsigned int token_line = 1;
029  static unsigned int last_token_line = 1;
030  static token current_token;
031
032  static const char *token_description(token value)
033  {
034      switch (value) {
035          case BEGIN_TOKEN: return "'BEGIN'";
036          case END_TOKEN:   return "'END'";
037          case READ_TOKEN:  return "'READ'";
038          case WRITE_TOKEN: return "'WRITE'";
039          case ID:           return "un identificateur";
040          case INTLITERAL:   return "un entier";
041          case REALLITERAL:  return "un reel";
042          case LPAREN:       return "'('";
043          case RPAREN:       return "')'";
044          case SEMICOLON:    return "';'";
045          case COMMA:        return "','";
046          case ASSIGNOP:     return "':='";
047          case PLUSOP:       return "'+'";
048          case MINUSOP:      return "'-'";
049          case SCANEOF:      return "la fin du fichier";
050      }
```

Annexe B - Code complet du TP2 - suite 1

Fichier TP2/tp2.c - lignes 51 à 100. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
051
052     return "un symbole inconnu";
053 }
054
055 static token token_from_name(const char *name)
056 {
057     static const char *const names[] = {
058         "BEGIN", "END", "READ", "WRITE", "ID", "INTLITERAL",
059         "REALLITERAL", "LPAREN", "RPAREN", "SEMICOLON", "COMMA",
060         "ASSIGNOP", "PLUSOP", "MINUSOP"
061     };
062     size_t index;
063
064     for (index = 0; index < sizeof names / sizeof names[0]; ++index) {
065         if (strcmp(name, names[index]) == 0)
066             return (token)index;
067     }
068
069     return SCANEOF;
070 }
071
072 /* Lit une ligne produite par le TP1 et restitue le token correspondant. */
073 static token scanner(void)
074 {
075     char line[512];
076     char token_name[32];
077     token value;
078     unsigned int line_number;
079
080     while (fgets(line, sizeof line, source_file) != NULL) {
081         if (strncmp(line, "LIGNE", 5) == 0 || strncmp(line, "-----", 5) == 0)
082             continue;
083
084         if (sscanf(line, "%u %31s %255[^\n]", &line_number,
085                    token_name, token_buffer) != 3) {
086             fprintf(stderr, "%s: erreur: ligne de tokens invalide: %s",
087                     source_name, line);
088             fclose(source_file);
089             exit(EXIT_FAILURE);
090         }
091
092         value = token_from_name(token_name);
093         if (value == SCANEOF) {
094             fprintf(stderr, "%s:%u: erreur: unite inconnue '%s'\n",
095                     source_name, line_number, token_name);
096             fclose(source_file);
097             exit(EXIT_FAILURE);
098         }
099
100         token_line = line_number;
```

Annexe B - Code complet du TP2 - suite 2

Fichier TP2/tp2.c - lignes 101 à 150. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
101         last_token_line = line_number;
102         return value;
103     }
104
105     token_buffer[0] = '\0';
106     token_line = last_token_line;
107     return SCANEOF;
108 }
109
110 /* Un token d'anticipation : consulter ne consomme jamais le token. */
111 static token next_token(void)
112 {
113     return current_token;
114 }
115
116 static void syntax_error(const char *expected)
117 {
118     fprintf(stderr, "%s:%u: erreur syntaxique : %s ; ",
119             source_name, token_line, expected);
120     if (next_token() == SCANEOF)
121         fputs("fin du fichier rencontree", stderr);
122     else
123         fprintf(stderr, "'%s' rencontre", token_buffer);
124     fputc('\n', stderr);
125     fclose(source_file);
126     exit(EXIT_FAILURE);
127 }
128
129 static void match(token expected)
130 {
131     if (next_token() != expected) {
132         char message[128];
133         if (expected == SCANEOF)
134             snprintf(message, sizeof message, "fin du fichier attendue");
135         else
136             snprintf(message, sizeof message, "%s attendu", token_description(expected));
137         syntax_error(message);
138     }
139
140     if (expected != SCANEOF)
141         current_token = scanner();
142 }
143
144 static void expression(void);
145 static void inst(void);
146
147 /* <add_op> ::= PLUSOP | MINUSOP */
148 static void add_op(void)
149 {
150     token tok = next_token();
```

Annexe B - Code complet du TP2 - suite 3

Fichier TP2/tp2.c - lignes 151 à 200. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
151     if (tok == PLUSOP || tok == MINUSOP)
152         match(tok);
153     else
154         syntax_error("operateur '+' ou '-' attendu");
155 }
156
157 /* <prim> ::= '(' <expr> ')' | ID | INTLITERAL | REALLITERAL */
158 static void prim(const char *expected)
159 {
160     switch (next_token()) {
161         case LPAREN:
162             match(LPAREN);
163             expression();
164             match(RPAREN);
165             break;
166         case ID:
167             match(ID);
168             break;
169         case INTLITERAL:
170             match(INTLITERAL);
171             break;
172         case REALLITERAL:
173             match(REALLITERAL);
174             break;
175         default:
176             syntax_error(expected);
177             break;
178     }
179 }
180
181 /* <expr> ::= <prim> { <add_op> <prim> } */
182 static void expression(void)
183 {
184     prim("identificateur, nombre ou expression entre parentheses attendu");
185     while (next_token() == PLUSOP || next_token() == MINUSOP) {
186         char message[128];
187         token op = next_token();
188         snprintf(message, sizeof message, "operande attendu apres %s",
189                  token_description(op));
190         add_op();
191         prim(message);
192     }
193 }
194
195 /* <id_list> ::= ID { ',' ID } : au moins un identificateur. */
196 static void id_list(void)
197 {
198     match(ID);
199     while (next_token() == COMMA) {
200         match(COMMA);
```

Annexe B - Code complet du TP2 - suite 4

Fichier TP2/tp2.c - lignes 201 à 250. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
201         match(ID);
202     }
203 }
204 
205 /* <expr_list> ::= <expr> { ',' <expr> } : au moins une expression. */
206 static void expr_list(void)
207 {
208     expression();
209     while (next_token() == COMMA) {
210         match(COMMA);
211         expression();
212     }
213 }
214 
215 /* <inst> ::= ID ':=' <expr> ';'
216  *          | READ '(' <id_list> ')' ';'
217  *          | WRITE '(' <expr_list> ')' ';'
218  */
219 static void inst(void)
220 {
221     switch (next_token()) {
222         case ID:
223             match(ID);
224             match(ASSIGNOP);
225             expression();
226             match(SEMICOLON);
227             break;
228         case READ_TOKEN:
229             match(READ_TOKEN);
230             match(LPAREN);
231             id_list();
232             match(RPAREN);
233             match(SEMICOLON);
234             break;
235         case WRITE_TOKEN:
236             match(WRITE_TOKEN);
237             match(LPAREN);
238             expr_list();
239             match(RPAREN);
240             match(SEMICOLON);
241             break;
242         default:
243             syntax_error("instruction attendue (affectation, READ ou WRITE)");
244             break;
245     }
246 }
247 
248 /* <inst_list> ::= <inst> { <inst> } : au moins une instruction. */
249 static void inst_list(void)
250 {
```

Annexe B - Code complet du TP2 - suite 5

Fichier TP2/tp2.c - lignes 251 à 297. Les numéros à gauche sont des repères de lecture ; ils ne font pas partie du code.

```
251     inst();
252     while (next_token() == ID || next_token() == READ_TOKEN ||
253            next_token() == WRITE_TOKEN)
254         inst();
255 }
256
257 /* <program> ::= BEGIN <inst_list> END */
258 static void program(void)
259 {
260     match(BEGIN_TOKEN);
261     inst_list();
262     match(END_TOKEN);
263 }
264
265 /* <system_goal> ::= <program> SCANEOF */
266 static void system_goal(void)
267 {
268     program();
269     match(SCANEOF);
270 }
271
272 int main(int argc, char *argv[])
273 {
274     if (argc != 2) {
275         fprintf(stderr, "Utilisation: %s <fichier-tokens>\n", argv[0]);
276         return EXIT_FAILURE;
277     }
278
279     source_name = argv[1];
280     source_file = fopen(source_name, "r");
281     if (source_file == NULL) {
282         fprintf(stderr, "%s: erreur: impossible d'ouvrir le fichier source\n",
283                 source_name);
284         return EXIT_FAILURE;
285     }
286
287     current_token = scanner();
288     system_goal();
289     if (ferror(source_file)) {
290         fprintf(stderr, "%s: erreur de lecture du fichier source\n", source_name);
291         fclose(source_file);
292         return EXIT_FAILURE;
293     }
294     fclose(source_file);
295     puts("Analyse syntaxique reussie: programme valide.");
296     return EXIT_SUCCESS;
297 }
```